

Response to the Editors and Reviewers

Manuscript TACO-2026-55

We sincerely thank the editors and the reviewers for their careful reading and constructive comments. We have revised the manuscript accordingly. For clarity, reviewer comments are reproduced in blue, our responses are in black, and the corresponding changes in the revised manuscript are shown in red.

Contents

1	Reviewer 1	2
1.1	Question 1	2
1.2	Question 2	3
1.3	Question 3	4
1.4	Question 4	5
1.5	Question 5	6
1.6	Question 6	7
1.7	Question 7	10
1.8	Question 8	11
1.9	Question 9	12
1.10	Question 10	13
1.11	Question 11	14
1.12	Question 12	16
2	Reviewer 2	18
2.1	Question 1	18
2.2	Question 2	20
2.3	Question 3	22
2.4	Question 4	24
2.5	Question 5	28
3	Reviewer 3	34
3.1	Question 1	34
3.2	Question 2	36
3.3	Question 3	38

1 Reviewer 1

1.1 Question 1

Some aspects of the architecture lack a clear justification. FluxArch uses 8 compute tiles, each containing 8 clusters with 16 cores each, but the rationale behind this organization is not explained. Was this hierarchy derived from design-space exploration or empirical evaluation? How would a different hierarchy (For example, 16 -> 8 -> 8 instead of 8 -> 8 -> 16) perform?

We sincerely thank you for your helpful and constructive suggestion.

- (1) **Hierarchy sizing.** Our hierarchy is selected bottom-up through empirical evaluation. We added hierarchy-configuration sweeps under a common requirement that every workload retain at least 80% scaling efficiency. The results select 16 cores per cluster and eight clusters per Compute Tile as the largest sharing domains satisfying this criterion across all five workloads, yielding a 128-core tile and a 1,024-core system.

The selected hierarchy and its link to the sizing study are summarized in **Section 4.1**.

Original (Section 4.1, page 9):

Sixteen FluxArch Cores form a FluxArch Cluster, which shares a cache and TLB. Eight FluxArch Clusters form a FluxArch Tile, which is equipped with a multi-channel memory controller optimized for bandwidth-intensive MCPT applications like XSBench. At the top level, the system comprises eight Compute Tiles and a centralized Switch Tile. As shown in Table 1, we carefully balanced cache capacity against the large database requirements of MCPT to optimize area. A key architectural departure in FluxArch is the omission of a shared L3 cache. Prior analysis indicates that expanding cluster-local L2 capacity offers superior performance-per-area and reduces coherence complexity compared to a monolithic L3. This design aligns with throughput-oriented architectures that prioritize L2 proximity for massive TLP, making FluxArch more GPU-like while retaining independent control flows.

Revised (Section 4.1, page 8):

We size the hierarchy bottom-up using an empirical criterion that every evaluated workload must retain at least 80% scaling efficiency. The sweeps in Section 6.6 select 16 cores per cluster and 8 clusters per Compute Tile, yielding a 128-core tile. Eight such tiles are interconnected through a centralized Switch Tile to form a 1,024-core system.

- (2) **Alternative hierarchy configurations.** We also added direct evaluations of the $16 \times 8 \times 8$ organization, together with $32 \times 4 \times 8$ and $32 \times 8 \times 4$, while holding the total core count at 1,024, the memory system at 32 channels, and every crossbar (XBar) at no more than 64 endpoints. The revised evaluation shows geometric-mean performance of 1.00, 0.96, 0.93, and 0.89 for these four organizations, respectively. The selected $8 \times 8 \times 16$ organization performs best overall because fewer cluster-bound MPI processes preserve communication locality for CORAL2-P1, CORAL2-P2, CTS2 and Homogeneous, while the alternatives benefit XSBench through greater aggregate cluster-local level-2 (L2) cache capacity and bandwidth.

The design-space constraints, bottom-up sweeps, and alternative-hierarchy trade-offs are explained in **Section 6.6**.

Added (Section 6.6, pages 19–20):

We first constrain the design space in three ways. We cap each XBar at 64 endpoints to avoid impractical arbitration, wiring, and timing complexity. We fix both the system scale at 1,024 cores and the memory system at 32 channels, so alternative hierarchies have identical compute and off-chip bandwidth resources. Within this space, we require every workload to retain at least 80% scaling efficiency and size the sharing domains from the bottom up. Fig. 17(a) varies the number of cores per cluster: 16 cores retain 81.46%–97.98% efficiency, whereas at 32 cores four workloads fall below the criterion. Fig. 17(b) then fixes 16-core clusters and varies the number of clusters per tile: eight clusters retain 81.02%–94.93%, whereas at 16 clusters only CTS2 remains above 80%. These sweeps establish 16 cores per cluster and eight clusters per tile as the largest per-level configurations that satisfy the criterion across all workloads, while smaller configurations remain viable alternatives.

We next compare $8 \times 8 \times 16$ against three representative alternatives that also satisfy these

constraints: $16 \times 8 \times 8$, $32 \times 4 \times 8$, and $32 \times 8 \times 4$, which progressively distribute the same 1,024 cores across more, smaller sharing domains. Fig. 17(c) reports geometric-mean normalized performance of 1.00, 0.96, 0.93, and 0.89 for these four configurations, respectively. The per-workload results reveal two opposing effects. For CORAL2-P1, CORAL2-P2, CTS2 and Homogeneous, performance declines as the number of cluster-bound MPI processes rises from 64 to 128, 128, and 256. More process boundaries increase communication overhead, which is material because particle exchange already accounts for 16.93%–31.80% of cycle-tracking time in Table 6(c). XSBench has no inter-process particle exchange and instead benefits from the higher aggregate capacity and bandwidth of 128, 128, and 256 cluster-local L2s. This sensitivity agrees with Table 6(a), where XSBench has the largest database, lowest L2 read hit rate, and highest normalized DRAM read traffic. Thus, $8 \times 8 \times 16$ best balances communication locality and aggregate L2 resources across the workload suite.

The per-level scaling sweeps and normalized alternative-hierarchy results are added in **Figure 17(a–c)**.

Added (Figure 17, page 19):

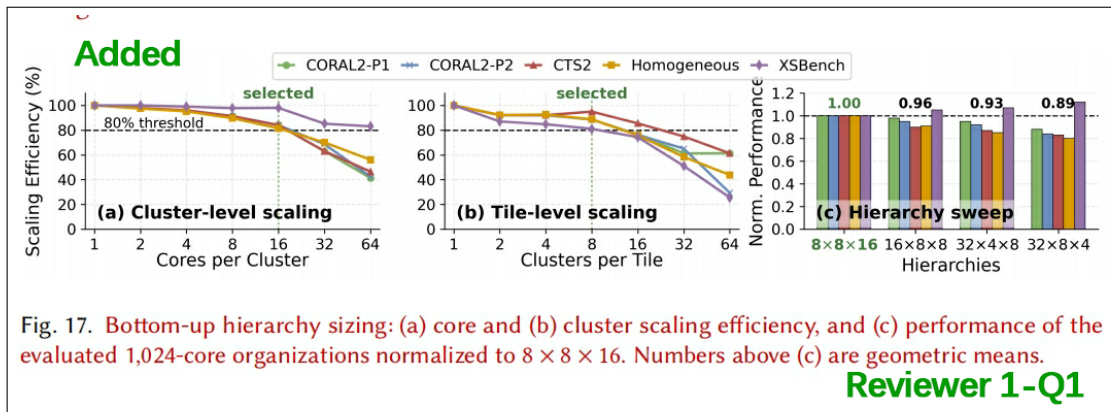


Fig. 17. Bottom-up hierarchy sizing: (a) core and (b) cluster scaling efficiency, and (c) performance of the evaluated 1,024-core organizations normalized to $8 \times 8 \times 16$. Numbers above (c) are geometric means.

Reviewer 1-Q1

The database sizes, L2 hit rates, and DRAM traffic supporting the XSBench explanation are reported in **Table 6(a)**.

Added (Table 6(a), page 18):

Table 6. Memory behavior, rank-level load balance, and inter-tile communication on the 1,024-core FluxArch system. Each MPI rank is bound to a 16-core cluster with a 4 MB shared L2. (a) Database footprint and memory behavior under LLC-aware mapping. (b) Rank-level work distribution and computation balance efficiency. (c) Peak directional TII utilization and particle-exchange time as a fraction of cycle-tracking time.

Added

Workload	(a)			(b)		(c)	
	DB/process (MiB)	L2 read hit rate	DRAM read (B/work)	Segment CV	Balance efficiency	Peak TII utilization (%)	Exchange Time
CORAL2-P1	0.318	92.07%	84.59	0.21%	97.72%	12.22	31.80%
CORAL2-P2	0.055	95.81%	140.81	0.16%	96.78%	28.15	26.59%
CTS2	0.318	91.98%	45.97	2.15%	95.45%	10.88	16.93%
Homogeneous	0.160	94.73%	200.82	5.53%	96.33%	19.73	30.00%
XSBench	241	64.58%	617.79	—	—	—	—

—: not reported or not applicable.

Reviewer 1-Q1

1.2 Question 2

The thread affinity and spatial decomposition strategies described in Section 5 may introduce load imbalance, causing some cores to handle significantly more work than others. The authors acknowledge this as a limitation and suggest dynamic partitioning as a future work direction, but it would still be valuable to at least quantify the resulting imbalance and its impact on performance.

Our static affinity and spatial decomposition assign one MPI rank to each cluster, so stochastic particle histories can create unequal work across the 64 ranks. We added a rank-level evaluation using both completed segments and accumulated tracking-kernel time rather than assuming perfect balance. The coefficient of variation (CV) of kernel time is only 1.17%–2.01%, balance efficiency is

95.45%–97.72%, and the resulting computation-side tail loss is at most 4.55% for the evaluated inputs. These results indicate that static affinity and spatial decomposition maintain balanced computation across clusters for the evaluated workloads at 1,024 cores, with limited computation-side loss from load imbalance.

The rank-level load-balance method and its computation-side impact are evaluated in **Section 6.5**.

Added (Section 6.5, pages 18–19):

Load Balance. Table 6(b) quantifies the concern that static affinity and spatial decomposition may assign unequal work to clusters. We record completed segments and accumulated tracking-kernel time separately for all 64 ranks. Segment-count CV ranges from 0.16% to 5.53%; because stochastic segments can follow different paths and incur different costs, count variation does not translate proportionally into execution-time variation. Kernel-time CV remains 1.17%–2.01%. We define balance efficiency as the mean rank-level kernel time divided by the maximum rank-level kernel time, since the slowest rank determines the computation tail. The resulting efficiency is 95.45%–97.72%, corresponding to an estimated computation-side tail loss of at most 4.55%. These results indicate that static affinity and spatial decomposition maintain balanced computation across clusters for the evaluated workloads at 1,024 cores, with limited computation-side loss from load imbalance.

The segment-count variation and balance efficiency are added in **Table 6(b)**.

Added (Table 6(b), page 18):

Table 6. Memory behavior, rank-level load balance, and inter-tile communication on the 1,024-core FluxArch system. Each MPI rank is bound to a 16-core cluster with a 4 MB shared L2. (a) Database footprint and memory behavior under LLC-aware mapping. (b) Rank-level work distribution and computation balance efficiency. (c) Peak directional TII utilization and particle-exchange time as a fraction of cycle-tracking time.

Added

Workload	(a)			(b)		(c)	
	DB/process (MiB)	L2 read hit rate	DRAM read (B/work)	Segment CV	Balance efficiency	Peak TII utilization (%)	Exchange Time
CORAL2-P1	0.318	92.07%	84.59	0.21%	97.72%	12.22	31.80%
CORAL2-P2	0.055	95.81%	140.81	0.16%	96.78%	28.15	26.59%
CTS2	0.318	91.98%	45.97	2.15%	95.45%	10.88	16.93%
Homogeneous	0.160	94.73%	200.82	5.53%	96.33%	19.73	30.00%
XSBench	241	64.58%	617.79	—	—	—	—

—: not reported or not applicable.

Reviewer 1-Q2

1.3 Question 3

A missing aspect of the experimental setup is the modeling of inter-tile communication. More details are needed to ensure reproducibility and properly contextualize the results.

Inter-tile communication is explicitly modeled rather than treated as zero-cost functional transfer. We expanded the evaluation methodology to specify that MPI messages traverse the guest network stack, Intel Gigabit Ethernet (IGbE) endpoints, and **DistEtherLink**-based Tile Interconnection Interfaces (TIIs) and therefore incur protocol processing, packetization, serialization, and modeled link latency. We also added per-TII traffic and end-to-end particle-exchange measurements: the busiest direction reaches 10.88%–28.15% of the 1-Gb/s link capacity, while traffic CV across the eight TIIs is 0.50%–2.42%. Thus, communication has a measurable execution cost, but TII saturation or traffic imbalance is not the bottleneck in these experiments.

The MPI transport path and communication-cost measurement method are specified in **Section 6.1**.

Added (Section 6.1, page 13):

Inter-Tile Communication Modeling. The eight Compute Tiles are connected to the Switch Tile through eight bidirectional Tile Interconnection Interfaces (TIIs). In distributed full-system **gem5**, each TII is modeled using a **DistEtherLink** and an IGbE endpoint, with the directional 1-Gb/s capacity in Table 1 and the modeled inter-tile latency. MPI messages traverse the guest network stack and these modeled links and are therefore subject to software protocol processing, packetization, link serialization, and inter-tile latency rather than zero-cost functional communication. We use the Ethernet-device counters to measure transmitted bytes and directional utilization at every TII.

Quicksilver’s cycle-tracking timers separately measure end-to-end particle-exchange time, including buffer handling, MPI communication and waiting, and receive-side particle management.

The measured TII utilization and traffic balance are analyzed in **Section 6.7**.

Added (Section 6.7, page 21):

Table 6(c) reports peak directional TII utilization and the particle-exchange share of cycle-tracking time. Across CORAL2-P1, CORAL2-P2, CTS2 and Homogeneous, FluxArch injects 1.37–1.46 bytes of inter-tile traffic per completed segment. The busiest directional TII reaches only 10.88%–28.15% of its modeled 1-Gb/s capacity. Bidirectional link traffic is also well balanced: its coefficient of variation across the eight TIIs is 0.50%–2.42%. Thus, the communication-driven spatial mapping avoids TII hotspots, and the evaluated links remain below saturation.

The peak directional TII utilization and particle-exchange share are reported in **Table 6(c)**.

Added (Table 6(c), page 18):

Table 6. Memory behavior, rank-level load balance, and inter-tile communication on the 1,024-core FluxArch system. Each MPI rank is bound to a 16-core cluster with a 4 MB shared L2. (a) Database footprint and memory behavior under LLC-aware mapping. (b) Rank-level work distribution and computation balance efficiency. (c) Peak directional TII utilization and particle-exchange time as a fraction of cycle-tracking time.

Added

Workload	(a)			(b)		(c)	
	DB/process (MiB)	L2 read hit rate	DRAM read (B/work)	Segment CV	Balance efficiency	Peak TII utilization (%)	Exchange Time
CORAL2-P1	0.318	92.07%	84.59	0.21%	97.72%	12.22	31.80%
CORAL2-P2	0.055	95.81%	140.81	0.16%	96.78%	28.15	26.59%
CTS2	0.318	91.98%	45.97	2.15%	95.45%	10.88	16.93%
Homogeneous	0.160	94.73%	200.82	5.53%	96.33%	19.73	30.00%
XSBench	241	64.58%	617.79	—	—	—	—

—: not reported or not applicable.

Reviewer 1-Q3

1.4 Question 4

In the experimental setup, there is no mention of random seed control or variability between runs, which are important details as MCPT is stochastic.

We use the same fixed workload-default pseudorandom seed for each workload across platforms to support reproducibility. Each experiment is repeated 10 times under exclusive-platform and stable-thermal conditions, and we report the arithmetic mean of the workload-native performance metric. Across the platform–workload configurations in Figure 13(a), the throughput coefficient of variation (CV) ranges from 0.1% to 6.2%, and from 0.1% to 0.4% for FluxArch. These values quantify run-to-run variability with the seed held fixed. We have added the repetition count and CV ranges to **Section 6.1**. The seed policy and reporting of run-to-run variability are clarified in **Section 6.1**.

Added (Section 6.1, page 14):

Reproducibility and Run-to-Run Variability. All platforms use fixed workload-default pseudorandom seeds (Table 4). Each experiment is repeated 10 times, and we report the arithmetic mean of the workload-native performance metric. Runs use exclusive platforms under stable thermal conditions. For Fig. 13(a), the throughput coefficient of variation (CV) is 0.1%–6.2% across platform–workload configurations and 0.1%–0.4% for FluxArch.

The workload seeds are specified in **Table 4**, which corresponds to **Table 2** in the original manuscript.

Original (Table 2, page 14):

Benchmark	Database	Nuclide	Reaction	Group/Gridpoint	nSteps	nBatches	nParticles(per thread)
CORAL2-P1	Multi-Group	20	9	230	10	10	32,768
CORAL2-P2	Multi-Group	10	3	230	10	10	32,768
CTS2	Multi-Group	20	9	230	10	10	32,768
Homogeneous	Multi-Group	10	9	230	10	10	32,768
EMCP	Point-Wise	35	3	7,863	10	10	1,024
XSBench	Conti-Energy	68	4	11,303	34	1	8,192

Revised (Table 4, page 14):

Table 4. Key parameters of benchmarks							
Benchmark	Database	Nuclide	Reaction	Group/ Gridpoint	nSteps/ nBatches	nParticles (thread)	Seed
CORAL2-P1(P1)	Multi-Group	20	9	230	10/10	32,768	1029384756
CORAL2-P2(P2)	Multi-Group	10	3	230	10/10	32,768	1029384756
CTS2	Multi-Group	20	9	230	10/10	32,768	1029384756
Homogeneous(Homo)	Multi-Group	10	9	230	10/10	32,768	1029384756
EMCP(EMCP)	Point-Wise	35	3	7,863	10/10	1,024	1029384756
XSBench(XS)	Conti-Energy	68	4	11,303	34/1	8,192	1070

Reviewer 1-Q4

1.5 Question 5

The authors describe how they measured the power for FluxArch but not for the baseline architectures. It is important to include such details for reproducibility and to ensure fair comparison (check the next cogent for the fairness).

The CPU and GPU baselines run natively, whereas FluxArch is evaluated through gem5 with McPAT and CACTI package-power modeling. We expanded the methodology to identify the measurement tools and align the accounting boundaries: `powerstat` records CPU/FT package power, `nvidia-smi` records GPU-board power, and off-package DDR5 is modeled separately where telemetry does not include it. This treatment includes co-packaged DDR4 or HBM only once and avoids omitting external DDR5 from the other platforms.

The native measurement tools, simulated power model, and measurement interval are specified in Section 6.1.

Added (Section 6.1, page 14):

Power and Area Modeling Methodology. Table 5(b) distinguishes the measured and simulated platforms and defines the power-accounting boundary. We execute all baselines natively. For Gold 6430 $\times 2$, EPYC 9654 $\times 2$, and FT 64-core, we collect on-package power with `powerstat` [31]; for NVIDIA A100 and NVIDIA H100, we record GPU-board power with `nvidia-smi` [46]. We report the mean power observed during each workload’s timed execution region. FluxArch is evaluated in gem5, with package power modeled using McPAT and CACTI.

References cited in this excerpt:

- [31] Colin Ian King. 2026. *powerstat: A Laptop Power Consumption Monitor*. Canonical Ltd. <https://manpages.ubuntu.com/manpages/jammy/man8/powerstat.8.html> Accessed: 2026-07-22.
- [46] NVIDIA Corporation. 2026. *NVIDIA System Management Interface (nvidia-smi)*. <https://docs.nvidia.com/deploy/nvidia-smi/> Accessed: 2026-07-22.

The platform-specific measurement and power-accounting boundaries are clarified in Table 5, which corresponds to original Table 3. Panel (a) revises the platform specifications, while panel (b) adds the evaluation methods and power-accounting boundaries.

Original (Table 3, page 14):

Table 3. Hardware parameters of platforms				
Platform	Specification	Memory	Technology	TDP
Intel Xeon Gold 6430 $\times 2$ [29]	2 Sockets, 64 Cores	16-Channel DDR5-4400	Intel 10nm	540 W
AMD EPYC 9654 $\times 2$ [2]	2 Sockets, 24 CCDs, 192 Cores	24-Channel DDR5-4800	TSMC 5nm	720 W
Phytium FT 64-core [49]	8 NUMA Nodes, 64 Cores	Co-packaged 8-channel DDR4	16nm	160 W
NVIDIA A100 SXM4 80GB [45]	108 SMs, 3,456 fp64 CUDA Cores	5 HBM2e, 2039 GB/s	TSMC 7nm	400 W
NVIDIA H100 PCIe [46]	114 SMs, 7,296 fp64 CUDA Cores	5 HBM2e, 1555 GB/s	TSMC 4nm	350 W
FluxArch	8 Tiles, 64 Clusters, 1,024 Cores	32-Channel DDR5-4800	22nm	-

Revised (Table 5, page 15):

Revised				
Table 5. Hardware and evaluation methodology of platforms.				
	Platform	Specification	Memory	Technology
(a)	Intel Xeon Gold 6430 $\times 2$ [26]	2 sockets, 64 cores	16-Channel DDR5-4400	Intel 10nm
	AMD EPYC 9654 $\times 2$ [1]	2 sockets, 24 CCDs, 192 cores	24-Channel DDR5-4800	TSMC 5nm
	Phytium FT 64-core [46]	8 NUMA nodes, 64 cores	Co-packaged 8-channel DDR4	16nm
	NVIDIA A100 SXM4 80GB [42]	108 SMs	5 HBM2e, 2039 GB/s	TSMC 7nm
	NVIDIA H100 PCIe [43]	114 SMs	5 HBM2e, 1555 GB/s	TSMC 4nm
	FluxArch	8 tiles, 64 clusters, 1,024 cores	32-Channel DDR5-4800	22nm
	Platform	Execution	Power source	Accounting boundary
(b)	Intel Xeon Gold 6430 $\times 2$ [26]	Native hardware	powerstat + DRAMPower	On-package + modeled DDR5
	AMD EPYC 9654 $\times 2$ [1]	Native hardware	powerstat + DRAMPower	On-package + modeled DDR5
	Phytium FT 64-core [46]	Native hardware	powerstat	On-package (co-packaged DDR4)
	NVIDIA A100 SXM4 80GB [42]	Native hardware	nvidia-smi	GPU board (including HBM)
	NVIDIA H100 PCIe [43]	Native hardware	nvidia-smi	GPU board (including HBM)
	FluxArch	gem5	McPAT + CACTI + DRAMPower	Modeled package + DDR5

Reviewer 1-Q5

1.6 Question 6

The authors mention that all reported energy metrics focus exclusively on on-chip power and that off-chip DRAM power is excluded. However, this makes the energy comparisons not very informative and, in cases, unfair. This is especially the case when comparing against architectures that have L3 cache. As FluxArch has no shared L3 cache, it is likely to have more misses than the baselines with L3 cache. Moreover, the baselines with L3 cache get unfairly penalized as their L3 power is included.

We agree that on-chip-only energy accounting can bias a memory-bound comparison, especially when the platforms expose different memory-packaging boundaries. We therefore changed the evaluation to include co-packaged DDR4/HBM through device telemetry and off-package DDR5 through a common traffic-based DRAMPower [10] model, with a $0.5 \times -2 \times$ sensitivity range for the modeled component. After this correction, the conservative process-scaled geometric-mean DRAM-inclusive energy-efficiency advantage remains $1.77 \times$ over H100 and $6.65 \times$ over Intel.

We now explain the choice of the $0.5 \times -2 \times$ range in **Section 6.1**. Ghose et al. [19] measured DDR3L read currents averaging 54.1% below datasheet values for one vendor and data-pattern-dependent read-power differences of up to 91.6%. Shi et al. [56] report that DDR4 calibration reduced DRAMPower energy estimates by roughly half for most evaluated SPEC2006 workloads. These results motivate testing sizable modeling discrepancies, but do not establish an error bound for our DDR5 configuration. We therefore select reciprocal factors of 0.5 and 2 to test both overestimation and underestimation, with package power fixed; the range is a sensitivity-analysis choice, not a statistical confidence interval.

For process-scaled energy-efficiency bounds, we independently vary only the scaled FluxArch package power by $\pm 30\%$ and each modeled off-package DRAM component over $0.5 \times -2 \times$, while keeping throughput and measured baseline package power fixed. We recompute the performance/W ratio at every endpoint combination and report its minimum and maximum; external DRAM power is not subject to process-node scaling or its 30% margin.

The power-accounting boundary is revised in **Section 6.1** to include DRAM consistently across platforms.

Revised (Section 6.1, page 14):

We report DRAM-inclusive system power and energy. Package power includes cores, caches, NoCs, and integrated memory controllers; for FT 64-core, NVIDIA A100, and NVIDIA H100, the main memory is co-packaged (DDR4 or HBM) and is consequently covered by platform-level package/device telemetry. In contrast, the DDR5 DIMMs of Gold 6430 $\times 2$ and EPYC 9654 $\times 2$, and the DDR5 channels assumed for FluxArch, are physically off-package and are not covered by package-power readings. We therefore add a separately modeled DRAM component to these platforms,

avoiding asymmetric accounting that would include HBM/DDR4 power for the co-packaged-memory platforms but omit DDR5 power for the other platforms.

The traffic-based DRAM model and factor-of-two sensitivity analysis are added in **Section 6.1**.

Added (Section 6.1, pages 14–15):

For the DRAM model, we use DRAMPower 5 [10] with DDR5 timing and worst-case current parameters from the JEDEC DDR5 standard [28] and the Micron MT60B2G8HB-48B DDR5-4800 x8 datasheet [42]. FluxArch is modeled with 32 single-rank DDR5-4800 channels, each containing eight x8 DRAM devices. The model includes active-standby background power, refresh power, and read/write transfer power. The external-memory energy of Gold 6430 $\times 2$ and EPYC 9654 $\times 2$ is estimated using the same traffic-based DDR5 model and workload-specific memory traffic.

Because the simulation does not expose DRAM command traces or row-buffer events, we use a traffic-based DRAM energy estimate rather than claiming command-level precision. For each FluxArch workload, we derive read/write traffic from the memory-system statistics and use the simulated execution time. Prior DRAM measurement and calibration studies report substantial discrepancies between datasheet-based estimates and measured consumption [19, 56]. Motivated by these findings, we select a $0.5\times\text{--}2\times$ range for the nominal modeled DRAM power of every off-package DDR5 platform (Gold 6430 $\times 2$, EPYC 9654 $\times 2$, and FluxArch). This range is a sensitivity-analysis choice, not an empirically established DDR5 error bound or a statistical confidence interval.

References cited in this excerpt:

- [10] Karthik Chandrasekar, Christian Weis, Yonghui Li, Sven Goossens, Matthias Jung, Omar Naji, Benny Akesson, Norbert Wehn, and Kees Goossens. [n. d.]. DRAMPower: Open-source DRAM Power and Energy Estimation Tool. <https://github.com/tukl-msd/DRAMPower>. Version 5.0, accessed July 2026.
- [28] JEDEC Solid State Technology Association. [n. d.]. DDR5 SDRAM Standard. Technical Report JESD79-5. JEDEC. <https://www.jedec.org/standards-documents/docs/jesd79-5>
- [42] Micron Technology, Inc. 2023. 16Gb DDR5 SDRAM Addendum: MT60B4G4, MT60B2G8, and MT60B1G16, Die Revision A. Micron Technology, Inc. <https://www.micron.com/products/memory/dram-components/ddr5-sdram> Rev. D, DDR5-4800 x8 specifications; the evaluated configuration is MT60B2G8HB-48B:A.
- [19] Saugata Ghose, Abdullah Giray Yağlıkçı, Raghav Gupta, Donghyuk Lee, Kais Kudrolli, William X. Liu, Hasan Hassan, Kevin K. Chang, Niladrish Chatterjee, Aditya Agrawal, Mike O'Connor, and Onur Mutlu. 2018. What Your DRAM Power Models Are Not Telling You: Lessons from a Detailed Experimental Study. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 2 (2018), 41 pages. doi:10.1145/3224419
- [56] Xinyu Shi, Dina Ali Abdelhamid, Thomas Ilsche, Saeideh Alinezhad Chamazcoti, Timon Evenblij, Mohit Gupta, and Francky Catthoor. 2025. Calibrating DRAMPower Model for HPC: A Runtime Perspective from Real-Time Measurements. *arXiv:2411.17960v3*. <https://arxiv.org/abs/2411.17960v3> Revised September 3, 2025.

The energy-efficiency comparison and conservative lower bounds are updated in **Section 6.2** to include DRAM power.

Original (Section 6.2, pages 14–15):

Iso-Process Energy Efficiency. As shown in Fig. 13(b), the energy efficiency advantage is even more pronounced. By comparing projected iso-process nodes, we isolate the architectural contributions. According to Section. 6.1, the power projection error is 30%. Therefore, we incorporate this margin into our energy efficiency evaluation, which is represented as error bars in our results. Without the workload orchestration, the energy efficiency of FluxArch, mainstream server CPUs and high-end GPUs is at odds with each other. With the orchestration enabled, optimizing data locality and minimizing cross-tile traffic, FluxArch demonstrates a superior geometric mean energy efficiency improvement from $3.98\times$ (vs. FT 64-core, $5.69\times$ discounted by 30%) to $9.84\times$ (vs. Gold 6430 $\times 2$, $14.06\times$ discounted by 30%) even accounting for the potential projection error of pessimistic assumption. We also report the energy efficiency of the raw, unscaled data based on the 22 nm technology node. Even in this case, FluxArch still achieves an energy efficiency improvement of $1.17\times$ (vs. NVIDIA H100) to $4.59\times$ (vs. Gold 6430 $\times 2$).

Revised (Section 6.2, pages 15–16):

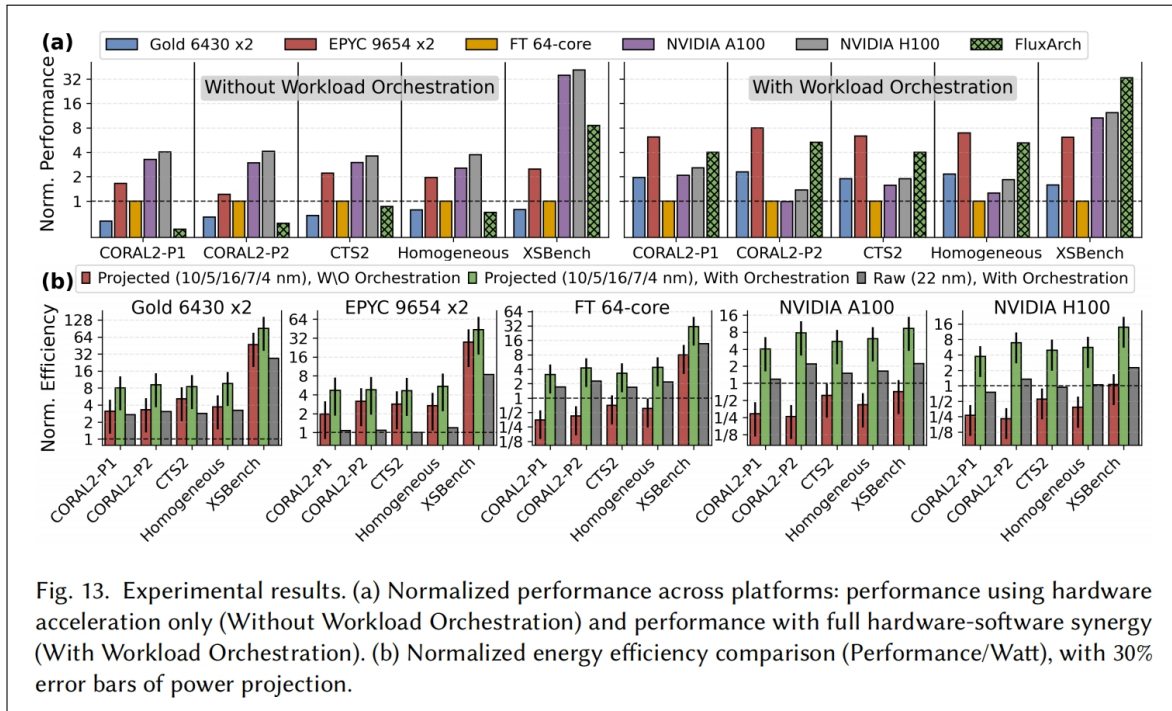
DRAM-Inclusive Energy Efficiency. Fig. 13(c,d) reports DRAM-inclusive power and energy efficiency under the sensitivity bounds defined in Section 6.1. Process-scaled results separate architectural effects from process scaling while accounting consistently for external memory.

In the process-scaled comparison, FluxArch improves geometric-mean DRAM-inclusive efficiency by $2.85\times$ over NVIDIA H100 to $10.58\times$ over Gold 6430 $\times 2$.

The sensitivity analysis preserves the process-scaled conclusion: across the combined DRAM and technology-scaling bounds, the geometric-mean improvement with orchestration remains $1.77\times$ over NVIDIA H100 and $6.65\times$ over Gold 6430 $\times 2$ at the conservative lower bound.

The power breakdown and energy-efficiency plots are updated in **Figure 13(c,d)** to include DRAM power and the corresponding sensitivity bounds. Panel (c) is newly added; panel (d) revises the original energy-efficiency plot.

Original (Figure 13, page 15):



Revised (Figure 13, page 16):

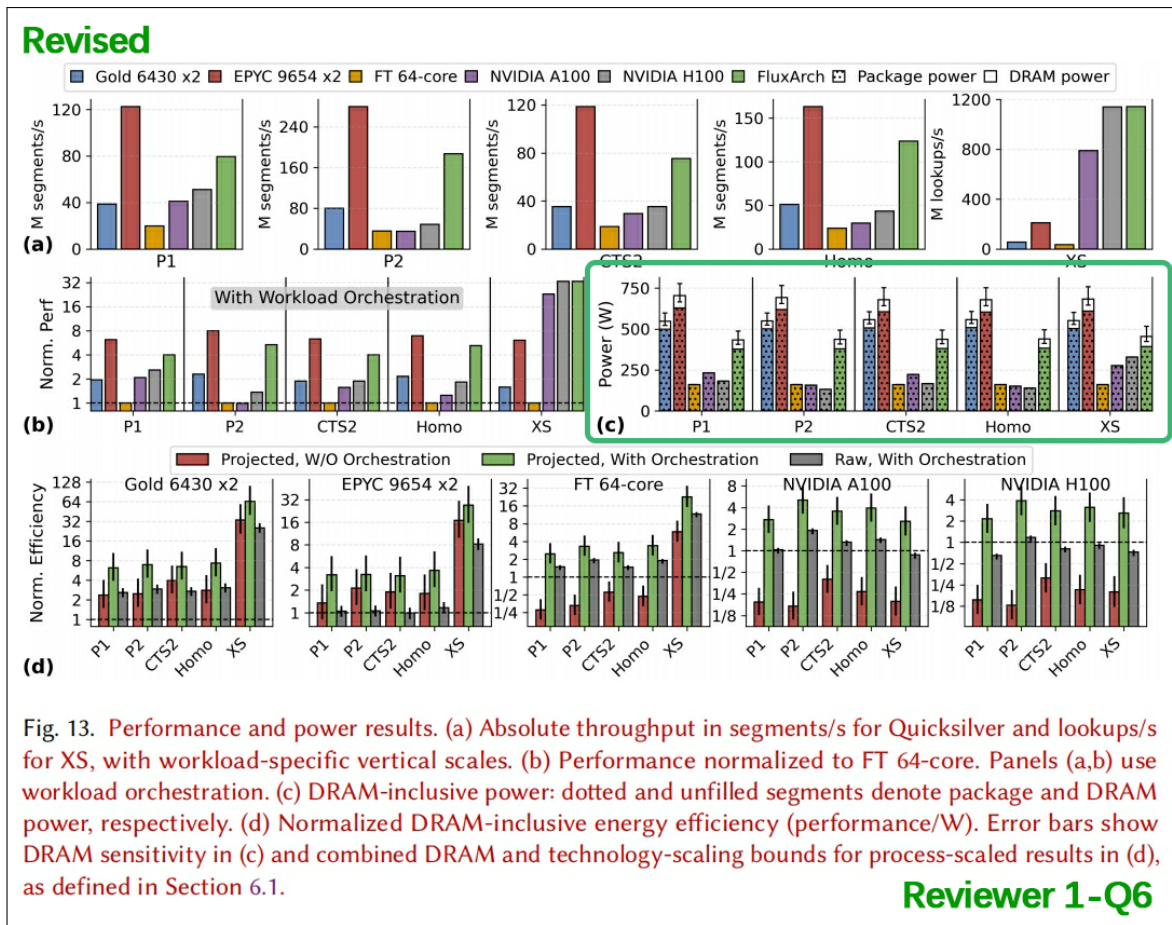


Fig. 13. Performance and power results. (a) Absolute throughput in segments/s for Quicksilver and lookups/s for XS, with workload-specific vertical scales. (b) Performance normalized to FT 64-core. Panels (a,b) use workload orchestration. (c) DRAM-inclusive power: dotted and unfilled segments denote package and DRAM power, respectively. (d) Normalized DRAM-inclusive energy efficiency (performance/W). Error bars show DRAM sensitivity in (c) and combined DRAM and technology-scaling bounds for process-scaled results in (d), as defined in Section 6.1.

Reviewer 1-Q6

1.7 Question 7

In Section 6.6 on scalability evaluation, the authors argue that the Gold 6430 $\times 2$ and FT 64-core outperform the proposed approach because their monolithic LLCs provide a unified, low-latency shared memory space that enables efficient inter-thread synchronization at lower core counts. However, no empirical data is provided to support this claim. Do the authors have measurements or analysis validating this explanation? This is particularly important since both systems significantly outperform FluxArch in most cases, and no experiments beyond 64 cores are possible to demonstrate whether FluxArch would eventually surpass them at larger scales (as they have no more cores).

We do not have controlled measurements that isolate LLC organization as the cause of the observed scaling difference. Cache organization, memory hierarchy, core microarchitecture, and synchronization overhead vary together across the evaluated platforms. The commercial baseline systems expose tightly coupled hardware configurations that cannot be independently reconfigured to change only LLC organization while holding the other factors constant. Our measurements therefore do not support the earlier LLC-based causal explanation, which we have removed. The revised discussion reports only the observed scaling trends within the evaluated ranges, without extrapolating Intel or FT beyond 64 cores. The scaling discussion is revised in **Section 6.6** to distinguish the measured trends from an unisolated LLC-based explanation and limit the CPU comparison to the evaluated range.

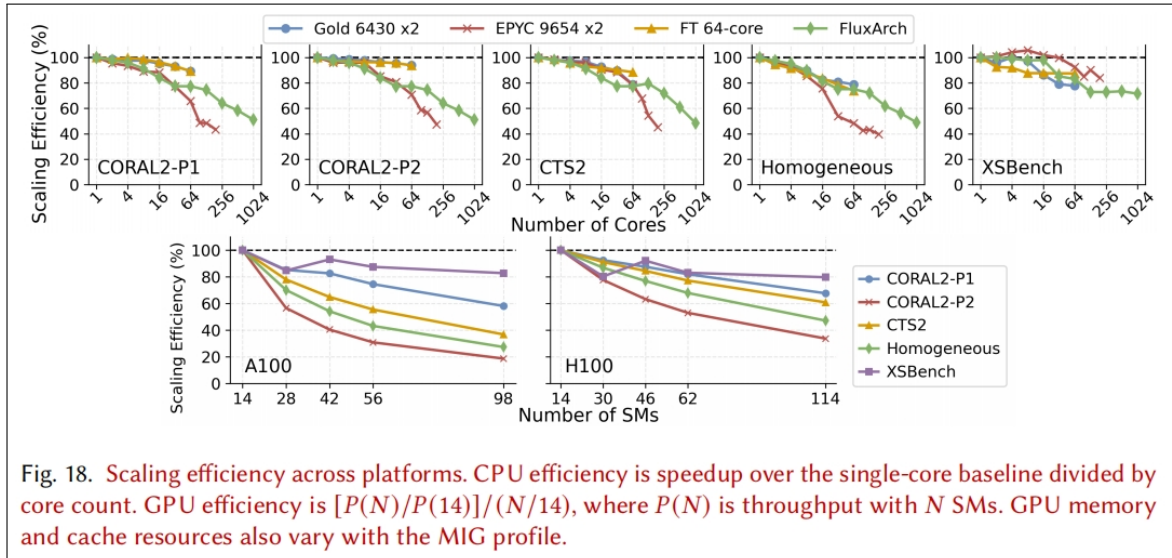
Original (Section 6.6, page 17):

Fig. 17 evaluates the scaling efficiency of FluxArch against three CPU platforms. In the small-scale regime (< 64 cores), FluxArch and EPYC 9654 $\times 2$ exhibit low efficiency. This is attributed to the architectural trade-off between monolithic and distributed Last-Level Caches (LLCs). Monolithic LLCs (Gold 6430 $\times 2$ /FT 64-core) provide a unified, low-latency shared memory pool that excels at handling inter-thread synchronization with a low core count. Conversely, the distributed LLCs in FluxArch and EPYC 9654 $\times 2$ incur higher coherence overhead when the workload is concentrated in fewer cores, as the shared memory programming model imposes a latency barrier on cross-cluster communication. However, in the large-scale regime (> 64 cores), FluxArch’s hierarchical design demonstrates superior efficiency, surpassing EPYC 9654 $\times 2$ across nearly all benchmarks as the system scales.

Revised (Section 6.6, page 20):

System-Level Scalability. Fig. 18 evaluates the scaling efficiency of FluxArch against three CPU platforms. At up to 64 cores, Gold 6430 $\times 2$ and FT 64-core retain higher scaling efficiency than FluxArch for most workloads. Cache organization, memory hierarchy, core microarchitecture, and synchronization overhead vary jointly across these platforms. The commercial baselines do not provide otherwise identical configurations differing only in LLC organization, so this cross-platform comparison characterizes system-level scaling rather than the isolated effect of LLC design. The evaluated Gold 6430 $\times 2$ and FT 64-core configurations cover at most 64 cores, limiting the comparison to this range. Within the measured range, FluxArch continues scaling to 1,024 cores and exceeds the measured EPYC 9654 $\times 2$ scaling efficiency across nearly all workloads at larger core counts.

Figure 18 (page 20):



1.8 Question 8

Why are no scalability results presented for GPUs? There are different ways to limit the GPU usage and emulate scalability. The authors are recommended to add such results or provide good reasons not to.

We added GPU scaling measurements by fixing each workload’s problem size and increasing the number of streaming multiprocessors (SMs) assigned to one GPU instance through NVIDIA Multi-Instance GPU (MIG). The five profiles provide 14, 28, 42, 56, and 98 SMs on A100, and 14, 30, 46, 62, and 114 SMs on H100. Each workload runs on one instance at a time. Efficiency is $E(N) = [P(N)/P(14)]/(N/14)$, where $P(N)$ is throughput with N SMs. Because memory capacity, cache, and memory bandwidth also depend on the profile, these results characterize scaling across GPU resource configurations. At 98 SMs on A100 and 114 SMs on H100, efficiency ranges from 18.62% to 82.73% and from 33.65% to 79.65%, respectively; XSBench scales best and CORAL2-P2 scales least efficiently on both GPUs.

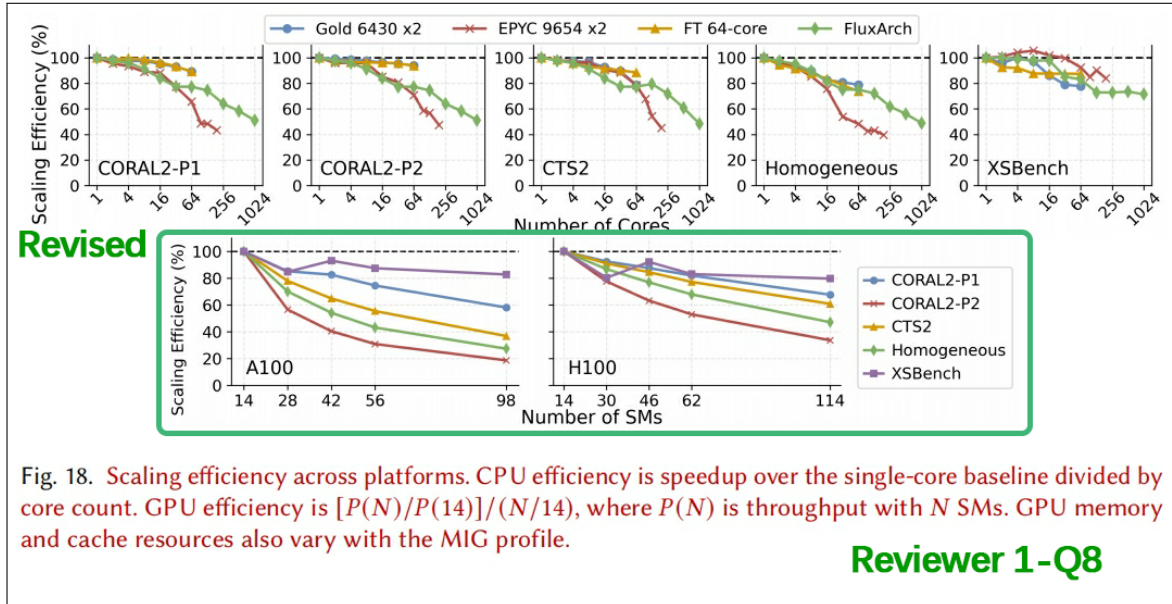
The MIG allocation method, normalization, and GPU scaling trends are described in **Section 6.6**.

Added (Section 6.6, page 20):

We additionally evaluate GPU scaling by keeping each workload’s problem size fixed and increasing the number of streaming multiprocessors (SMs) assigned to one GPU instance. NVIDIA Multi-Instance GPU (MIG) partitions a physical GPU into isolated instances with dedicated compute and memory resources. We use five MIG profiles to allocate 14–98 SMs on NVIDIA A100, and 14–114 SMs on NVIDIA H100. The cache and memory-bandwidth allocations vary with the profile, so these measurements reflect scaling across GPU resource configurations. At 98 SMs on NVIDIA A100 and 114 SMs on NVIDIA H100, efficiency ranges from 18.62% to 82.73% and from 33.65% to 79.65%, respectively. XSBench scales best on both GPUs, while CORAL2-P2 scales least efficiently.

The GPU scaling results are added in **Figure 18**, with GPU scale labeled by the actual SM counts. The CPU panels are revised from the original figure, while the GPU panels are newly added.

Revised (Figure 18, page 20):



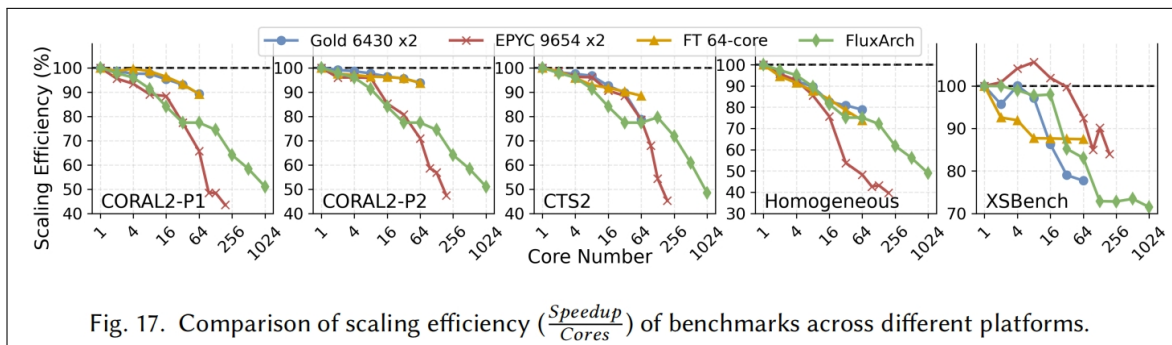
1.9 Question 9

The scalability behaviour of the XSBench benchmark is an outlier. The authors are recommended to discuss the reason behind this. Are there certain generalizable characteristics in this benchmark that affect FluxArch and EPYC scaling?

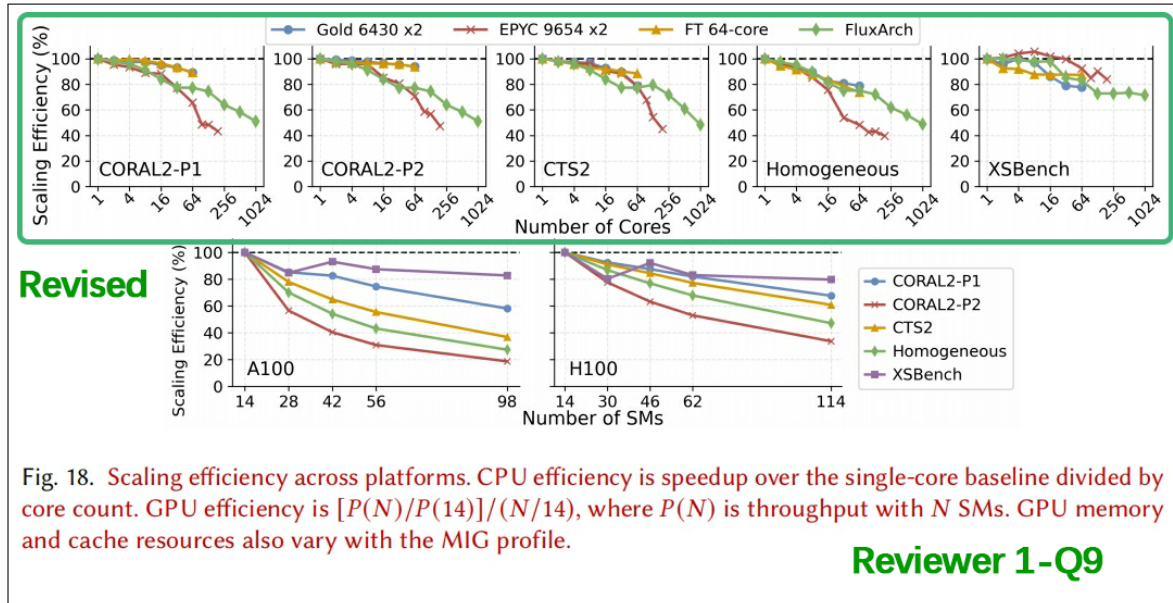
We revised **Figure 18** for clarity and aligned the y-axis scales across panels.

XSBench is an outlier in the favorable direction: it retains approximately 71% scaling efficiency at 1,024 cores, compared with approximately 48%–52% for the other FluxArch workloads. On EPYC, XSBench similarly retains approximately 84% efficiency at 192 cores, compared with approximately 40%–47% for the four Quicksilver workloads. EPYC and FluxArch both organize cores into local cache-sharing groups—CCDs sharing L3 and clusters sharing L2, respectively. Under our LLC-aware process mapping, XSBench’s independent lookups require no inter-rank particle-state exchange as execution expands across these groups, unlike Quicksilver. This favorable scaling does not imply full cache residency: its 241-MiB database exceeds the 4-MiB cluster L2 on FluxArch, and the FluxArch measurements in Table 6(a) report the lowest L2 read hit rate and highest normalized DRAM read traffic among the evaluated workloads. We clarified this shared workload characteristic and its relationship to the two platforms’ cache-sharing groups in **Section 6.6**.

Original (Figure 17, page 18):



Revised (Figure 18, page 20):



The favorable XSBench scaling is explained in **Section 6.6** through its independent lookups and absence of inter-rank particle exchange.

Added (Section 6.6, page 20):

XSBench retains approximately 71% scaling efficiency at 1,024 cores, compared with approximately 48%–52% for the other FluxArch workloads. EPYC and FluxArch both organize cores into local cache-sharing groups—CCDs sharing L3 and clusters sharing L2, respectively. Under our LLC-aware process mapping, XSBench’s independent lookups require no inter-rank particle-state exchange as execution expands across these groups, unlike Quicksilver. This favorable scaling does not imply full cache residency: the FluxArch measurements in Table 6(a) show that XSBench has the largest database, the lowest L2 read hit rate, and the highest normalized DRAM read traffic among the evaluated workloads.

The discussion also relates this behavior to the memory measurements in **Table 6(a)**.

Added (Table 6(a), page 18):

Table 6. Memory behavior, rank-level load balance, and inter-tile communication on the 1,024-core FluxArch system. Each MPI rank is bound to a 16-core cluster with a 4 MB shared L2. (a) Database footprint and memory behavior under LLC-aware mapping. (b) Rank-level work distribution and computation balance efficiency. (c) Peak directional TII utilization and particle-exchange time as a fraction of cycle-tracking time.

Added

Workload	(a)			(b)		(c)	
	DB/process (MiB)	L2 read hit rate	DRAM read (B/work)	Segment CV	Balance efficiency	Peak TII utilization (%)	Exchange Time
CORAL2-P1	0.318	92.07%	84.59	0.21%	97.72%	12.22	31.80%
CORAL2-P2	0.055	95.81%	140.81	0.16%	96.78%	28.15	26.59%
CTS2	0.318	91.98%	45.97	2.15%	95.45%	10.88	16.93%
Homogeneous	0.160	94.73%	200.82	5.53%	96.33%	19.73	30.00%
XSBench	241	64.58%	617.79	—	—	—	—

—: not reported or not applicable.

Reviewer 1-Q9

1.10 Question 10

The authors primarily evaluate CPUs and GPUs, but do not discuss alternative architectures such as FPGAs, CGRAs, or specialized manycore systems like Graphcore. These platforms may help mitigate some of the bottlenecks identified for CPUs and GPUs. In addition, prior work has explored FPGA-based approaches for Monte Carlo simulations. A comparison (discussion, not necessarily a complete evaluation) would strengthen the paper and provide a more complete perspective on the proposed architecture’s position in the design space.

We agree that FPGA-based Monte Carlo transport acceleration is relevant prior work and was insufficiently discussed in the original manuscript. We now explicitly discuss Gokhale et al.’s FPGA pipeline for ray–surface intersection in Monte Carlo radiative heat transfer and Luu et al.’s FPGA implementation of MCML photon transport. The comparison in **Section 8** is anchored in the boundary of specialization: these works specialize a transport loop or a particular photon-transport pipeline, whereas FluxArch retains instruction-driven MIMD transport control and integrates selected FP64 arithmetic kernels into each core. Their application-specific datapaths do not establish the cache/DRAM and communication organization used here for database-intensive neutron transport. We explain how FluxArch combines core-local acceleration with shared-cache reuse and topology-aware MPI/OpenMP orchestration, rather than suggesting that FPGAs cannot handle stochastic control. We also distinguish CGRA spatial mapping from instruction-driven control and Graphcore’s tile-local memory/exchange organization from FluxArch’s cache/DRAM hierarchy. The discussion compares design choices, not speedup values obtained under different physical models and workloads. The architectural differences from FPGAs, CGRAs, and the Graphcore IPU, together with the limits of a quantitative comparison, are clarified in **Section 8**.

Added (Section 8, pages 22–23):

Alternative Programmable Architectures. Gokhale et al. [21] accelerate Monte Carlo radiative heat transfer with an FPGA ray–surface intersection pipeline; Luu et al. [37] implement MCML photon transport through multilayer tissue as a dedicated FPGA pipeline. The distinction is the boundary of specialization: these designs specialize a transport loop or photon-transport pipeline, whereas FluxArch retains instruction-driven MIMD transport control and specializes selected FP64 arithmetic kernels within each core. Their datapaths target the geometry and interaction models of their respective applications. FluxArch couples its core-local Engines with cache- and DRAM-backed database access and topology-aware MPI/OpenMP orchestration, addressing data reuse and inter-process communication alongside arithmetic acceleration. CGRAs likewise expose configurable spatial datapaths, with computation mapped onto processing elements and their interconnect [12]; FluxArch instead leaves dynamic transport control in programmable cores. The Graphcore IPU is also MIMD, but its tile-local memory and explicit exchange [40] contrast with FluxArch’s cache/DRAM hierarchy and core-local Engine. Differences in physical models, workloads, and evaluation conditions preclude a direct comparison of the cited speedup values.

References cited in this excerpt:

- [21] Maya Gokhale, Janette Frigo, Christine Ahrens, Justin L. Tripp, and Ron Minnich. 2004. Monte Carlo Radiative Heat Transfer Simulation on a Reconfigurable Computer. In *Field Programmable Logic and Application*. Springer, Berlin, Heidelberg, 95–104. doi:10. 1007/978-3-540-30117-2_12
- [37] Jason Luu, Keith Redmond, William Chun Yip Lo, Paul Chow, Lothar Lilge, and Jonathan Rose. 2009. FPGA-based Monte Carlo Computation of Light Absorption for Photodynamic Cancer Therapy. In *2009 17th IEEE Symposium on Field Programmable Custom Computing Machines*. IEEE, Napa, CA, USA, 157–164. doi:10. 1109/FCCM. 2009. 24
- [12] Kiyoungh Choi. 2011. Coarse-Grained Reconfigurable Array: Architecture and Application Mapping. *IPSS Transactions on System LSI Design Methodology* 4 (2011), 31–46. doi:10. 2197/ipsjtsldm. 4. 31
- [40] Samuel Maddrell-Mander, Lakshan Ram Madhan Mohan, Alexander Marshall, Daniel O’Hanlon, Konstantinos Petridis, Jonas Rademacker, Victoria Rege, and Alexander Titterton. 2021. Studying the Potential of Graphcore IPUs for Applications in Particle Physics. *Computing and Software for Big Science* 5, 1 (2021), 8. doi:10. 1007/s41781-021-00057-z

1.11 Question 11

The authors mentioned related architectures are using specialized, area-efficient designs (Fu et al [22] and Zhang et al. [72]). However, there is no comparison with these approaches. In the related work section, the authors mention briefly how FluxArch qualitatively improves over these works but do not provide any quantitative comparisons. The authors are recommended to include a more detailed comparison against the mentioned approaches as this would strengthen the work considerably.

Fu et al. and Zhang et al. propose the specialized MCPT architectures most closely related to FluxArch. We added a detailed comparison in **Table 7**, covering core design, hierarchy and system

scale, cache and memory organization, evaluation methodology, workloads, and the quantitative results reported by each work.

We also explain in **Section 8** why these published values cannot be compared directly. Fu et al. report performance, performance/W, and performance/area relative to a modeled four-core Cortex-A15 system; Zhang et al. report scaling efficiency relative to their own single-core configuration. FluxArch’s performance and energy-efficiency ratios instead use our CPU/GPU baselines. Workload inputs, system resources, and accounting boundaries also differ substantially. The table therefore preserves each result’s original baseline rather than converting these values into cross-architecture speedups.

Reimplementing the designs in a common framework would not, by itself, establish a fair comparison. Although the papers provide major architectural parameters, their descriptions do not fully specify all implementation and modeling choices needed for a unified evaluation. In particular, Zhang et al. do not report a process node or a power/area model. Reconstructing customized simulator configurations and adding missing power/area models would require assumptions that could affect the results and confound architectural differences with reconstruction choices.

The comparison with Fu et al. and Zhang et al. is revised in **Section 8** to clarify the architectural differences.

Original (Section 8, page 19):

Recent microarchitectural studies have shifted toward specialized, area-efficient designs. Fu et al. pioneered the use of clustered small cores to improve energy-to-area ratios for MCPT workloads, a foundation subsequently extended by Zhang et al. to a 128-core cluster with optimized parallelization. Our FluxArch system achieves a significant leap forward by: (1) scaling the architecture to 1,024 cores using a hierarchical chiplet-based design; (2) integrating a dedicated execution engine to offload dominant kernels; and (3) proposing a topology-aware hardware-software co-design that transcends the scalability limits of prior clustered small-core architectures.

Revised (Section 8, page 22):

Specialized MCPT Architectures. Recent microarchitectural studies have shifted toward specialized, area-efficient designs. **Table 7 compares FluxArch with Fu et al. [18] and Zhang et al. [63] in core design, hierarchy, cache and memory organization, evaluation methodology, workloads, and author-reported results.** Fu et al. use simplified in-order cores to improve area and power efficiency, while Zhang et al. scale to 128 cores through 32-core cache-sharing clusters and OpenMP dynamic scheduling. FluxArch combines a 1,024-core hierarchy with a core-local Engine and topology-aware spatial mapping.

References cited in this excerpt:

- [18] *Siqing Fu, Tiejun Li, Jianmin Zhang, Sheng Ma, and Sheng Liu. 2021. MiniMCTAD: Minimalist Monte Carlo Transport Architecture Design. In 19th IEEE International Symposium on Parallel and Distributed Processing with Applications.*
- [63] *L. Zhang, T.-J. Li, and J.-M. Zhang. 2023. A 128-core scalable architecture for Monte Carlo application. Computer Engineering & Science / Jisuanji Gongcheng yu Kexue (2023), 590–598. Issue 4.*

The limits of comparing published results and reconstructing prior designs fairly are added in **Section 8**.

Added (Section 8, page 22):

The reported results use substantially different experimental conditions, including workload inputs, core counts, cache and memory configurations, reference baselines, and power-accounting boundaries. Fu et al. report performance, performance/W, and performance/area relative to a modeled four-core Cortex-A15 system. Zhang et al. report scaling efficiency relative to their own single-core configuration. FluxArch’s performance and energy-efficiency ratios use the evaluated CPU/GPU platforms as baselines, whereas its scaling efficiency is relative to one core. Thus, Table 7 places each work’s results in context rather than establishing cross-architecture speedups or an efficiency ranking.

A unified reimplementing would also require resolving implementation and modeling choices not fully specified in the papers, rather than merely matching their listed hardware parameters. In particular, Zhang et al. do not report a process node or a power/area model. Reconstructing the customized simulator configurations and supplying missing power/area models would introduce additional assumptions that could affect the results, confounding architectural differences with reconstruction choices and weakening the fairness of a direct comparison.

The detailed architectural parameters and author-reported results, with their original comparison baselines, are added in **Table 7**.

Added (Table 7, page 22):

Added			
Table 7. Quantitative comparison with prior specialized MCPT architectures. Reported improvements use each work’s own workload configuration and baseline and are not cross-paper normalized speedups.			
Work	Fu et al.	Zhang et al.	FluxArch
Core	MiniMCTAD @ 2.2 GHz	HPI @ 2.2 GHz	FluxArch Core + dedicated Engine @ 2GHz
Hierarchy and Scale	1×8	4×32	$8 \times 8 \times 16$
Cache	512 kB shared L2	8 MB shared L2/cluster	4 MB shared L2/cluster
Memory	8-channel HBM2	8-channel HBM2	32-channel DDR5
Methodology	gem5+McPAT,28nm	gem5	gem5+McPAT+CACTI+DRAMPower,22nm
Benchmark	Quicksilver default	Quicksilver default	CORAL2-P1,CORAL2-P2,CTS2,Homogeneous,XSBench
Author-reported quantitative result	Relative to 4 Cortex-A15: 1.416x performance 4.56x perf. efficiency 3.01x area efficiency	70.1% scaling efficiency	Relative to various platforms: 1.02x - 6.83x performance 2.85x–10.58x energy efficiency (nominal process-scaled, DRAM-inclusive) 48% - 71% scaling efficiency

Reviewer 1-Q11

1.12 Question 12

The authors are recommended to add references to the first paragraphs of the introduction for readers interested in more details.

We sincerely thank you for your helpful and constructive suggestion. We added the corresponding references.

References supporting the introductory motivation and background claims are added in **Section 1**.

Original (Section 1, page 1):

Monte Carlo Particle Transport (MCPT) is the authoritative standard for simulating interactions between particles and material, widely applied in critical fields such as nuclear reactor safety and medical radiation therapy. Its core philosophy involves performing independent and repeated random simulations of billions of particles and estimating macroscopic physical quantities, such as particle penetration probability and energy deposition distribution, based on statistical methods. Since each particle is statistically independent, MCPT theoretically possesses rich parallel potential.

However, in practice, each particle’s transport is driven by random sampling, which determines its energy state, reaction type (e.g., fission, capture), and interaction position via probability distributions. This stochastic nature renders particle execution paths highly unpredictable, thereby introducing extensive random control flow. As particles propagate, they must frequently access multi-dimensional probability databases based on their current energy and reaction states to perform interpolation and physical calculations. Since particles traverse highly dissimilar energy regions in state space, successive steps often access widely separated entries in the probability tables, resulting in significant spatial irregularity in memory access.

Consequently, the MCPT workload simultaneously exhibits Massive Parallelism, Random Control Flow, and Irregular Memory Access—a combination of characteristics that poses extreme architectural challenges, resulting in severe architectural mismatches on modern general-purpose computing platforms.

Revised (Section 1, page 1):

Monte Carlo Particle Transport (MCPT) is a widely used high-fidelity method for simulating interactions between particles and materials, with applications including nuclear reactor analysis and medical radiation treatment planning [11, 22, 38, 47, 51]. Its core philosophy involves performing independent and repeated random simulations of billions of particle histories and estimating macroscopic physical quantities, such as particle penetration probability and energy deposition distributions, using statistical methods [22, 38]. Since particle histories are statistically independent, MCPT theoretically possesses abundant parallelism [7, 8].

However, in practice, each particle’s transport is driven by random sampling, which determines its energy state, reaction type (e.g., fission, capture), and interaction position via probability distributions. This stochastic nature renders particle execution paths highly unpredictable, thereby

introducing extensive random control flow. As particles propagate, they must frequently access multi-dimensional probability databases based on their current energy and reaction states to perform interpolation and physical calculations. Since particles traverse highly dissimilar energy regions in state space, successive steps often access widely separated entries in the probability tables, resulting in significant spatial irregularity in memory access [7, 8, 39].

Consequently, the MCPT workload simultaneously exhibits Massive Parallelism, Random Control Flow, and Irregular Memory Access—a combination of characteristics that poses substantial architectural challenges on modern general-purpose computing platforms [7, 8, 39].

References cited in this excerpt:

- [7] Ryan C Bleile. 2021. *Enhancing Monte Carlo Particle Transport for Modern Many-Core Architectures*. Ph. D. Dissertation. University of Oregon.
- [8] Patrick S Brantley, Ryan C Bleile, Shawn A Dawson, NA Gentile, M Scott McKinley, Matthew J O'Brien, Michael M Pozulp, David F Richards, David E Stevens, Jonathan A Walsh, et al. 2017. *LLNL Monte Carlo transport research efforts for advanced computing architectures*. Technical Report. Lawrence Livermore National Lab.(LLNL), Livermore, CA (United States).
- [11] Yujie Chi, Keith Schubert, Andreu Badal, and Emilie Roncali. 2025. *Review of GPU-based Monte Carlo simulation platforms for transmission and emission tomography in medicine*. *Physics in Medicine & Biology* 70, 17 (2025), 17TR01.
- [22] Alireza Haghighat. 2020. *Monte Carlo methods for particle transport*. Crc Press.
- [38] Iván Lux. 2018. *Monte Carlo particle transport methods*. CRC press.
- [39] Donghui Ma, Bo Yang, Qingyang Zhang, Jie Liu, and Tiejun Li. 2021. *Evaluation of single-node performance of parallel algorithms for multigroup Monte Carlo particle transport methods*. *Frontiers in Energy Research* 9 (2021), 705823.
- [47] T.M. Pandya, S.R. Johnson, T.M. Evans, G.G. Davidson, S.P. Hamilton, and A.T. Godfrey. 2016. *Implementation, capabilities, and benchmarking of Shift, a massively parallel Monte Carlo radiation transport code*. *J. Comput. Phys.* (2016), 239–272. Issue No.C.
- [51] Paul K Romano and Benoit Forget. 2013. *The OpenMC Monte Carlo particle transport code*. *Annals of Nuclear Energy* (2013), 274–281. Issue C.

2 Reviewer 2

2.1 Question 1

Clarify measured vs. simulated and validate the model. State whether each baseline is real hardware or gem5, and validate FluxArch against a fabricated reference—the headline $6.83\times/9.84\times$ claims depend on this.

We sincerely thank you for your helpful and constructive suggestion. We agree that the evaluation must distinguish native measurements from simulation and provide direct evidence for the modeled components.

- (1) **Measured versus simulated results.** The Intel, AMD, FT, A100, and H100 baselines are executed and measured on native hardware, whereas FluxArch performance is simulated in gem5 and its package power and area are modeled with McPAT and CACTI. We revised the platform description so that the measurement and modeling boundary is explicit for every platform. The native results are hardware measurements, while the FluxArch results are calibrated simulation estimates.

The distinction between native measurements and modeled results is clarified in **Section 6.1**. The platform-specific evaluation and power-accounting methods are specified in **Table 5**, replacing original **Table 3**. Panel (a) revises the platform specifications, while panel (b) adds the evaluation methods and power-accounting boundaries.

Original (Table 3, page 14):

Table 3. Hardware parameters of platforms				
Platform	Specification	Memory	Technology	TDP
Intel Xeon Gold 6430 $\times 2$ [29]	2 Sockets, 64 Cores	16-Channel DDR5-4400	Intel 10nm	540 W
AMD EPYC 9654 $\times 2$ [2]	2 Sockets, 24 CCDs, 192 Cores	24-Channel DDR5-4800	TSMC 5nm	720 W
Phytium FT 64-core [49]	8 NUMA Nodes, 64 Cores	Co-packaged 8-channel DDR4	16nm	160 W
NVIDIA A100 SXM4 80GB [45]	108 SMs, 3,456 fp64 CUDA Cores	5 HBM2e, 2039 GB/s	TSMC 7nm	400 W
NVIDIA H100 PCIe [46]	114 SMs, 7,296 fp64 CUDA Cores	5 HBM2e, 1555 GB/s	TSMC 4nm	350 W
FluxArch	8 Tiles, 64 Clusters, 1,024 Cores	32-Channel DDR5-4800	22nm	-

Revised (Table 5, page 15):

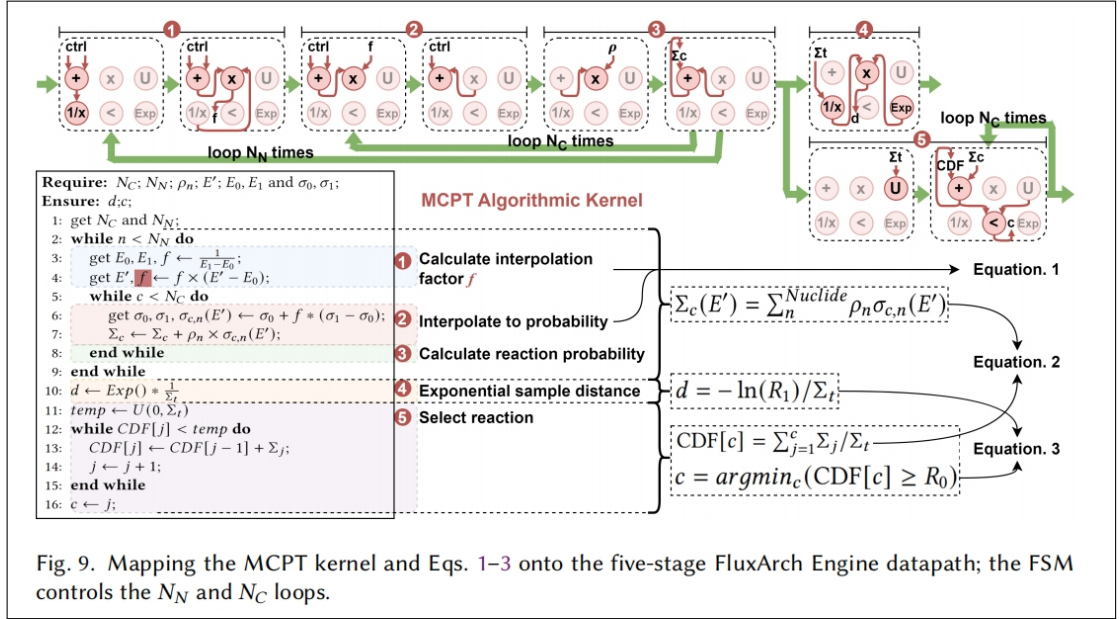
Table 5. Hardware and evaluation methodology of platforms.				
Revised				
Platform	Specification	Memory	Technology	
Intel Xeon Gold 6430 $\times 2$ [26]	2 sockets, 64 cores	16-Channel DDR5-4400	Intel 10nm	
AMD EPYC 9654 $\times 2$ [1]	2 sockets, 24 CCDs, 192 cores	24-Channel DDR5-4800	TSMC 5nm	
(a) Phytium FT 64-core [46]	8 NUMA nodes, 64 cores	Co-packaged 8-channel DDR4	16nm	
NVIDIA A100 SXM4 80GB [42]	108 SMs	5 HBM2e, 2039 GB/s	TSMC 7nm	
NVIDIA H100 PCIe [43]	114 SMs	5 HBM2e, 1555 GB/s	TSMC 4nm	
FluxArch	8 tiles, 64 clusters, 1,024 cores	32-Channel DDR5-4800	22nm	
Platform	Execution	Power source	Accounting boundary	
Intel Xeon Gold 6430 $\times 2$ [26]	Native hardware	powerstat + DRAMPower	On-package + modeled DDR5	
AMD EPYC 9654 $\times 2$ [1]	Native hardware	powerstat + DRAMPower	On-package + modeled DDR5	
(b) Phytium FT 64-core [46]	Native hardware	powerstat	On-package (co-packaged DDR4)	
NVIDIA A100 SXM4 80GB [42]	Native hardware	nvidia-smi	GPU board (including HBM)	
NVIDIA H100 PCIe [43]	Native hardware	nvidia-smi	GPU board (including HBM)	
FluxArch	gem5	McPAT + CACTI + DRAMPower	Modeled package + DDR5	

Reviewer 2-Q1

- (2) **Model validation.** We added six controlled comparisons against RK3562 Cortex-A53 silicon for the core and cache/DRAM paths and derived Engine latency directly from fixed-latency RTL operators and loop bounds. The silicon comparison yields matching checksums, 6.7% mean absolute percentage error, and 13.5% maximum absolute percentage error; the Engine expression is derived from the implementation. This establishes implementation traceability, not an independent statistical calibration between two timing models. We describe these results as calibrated simulation evidence while clearly retaining the limitation that the complete 1,024-core system has not been fabricated.

The deterministic Engine latency is derived from RTL operator timing and loop bounds in **Section 4.2**.

Figure 9 (page 9):



Added (Section 4.2, pages 9–10):

Here, N_N denotes the number of nuclides and N_C the number of reaction channels per nuclide; they determine the outer and inner loop bounds, respectively.

Deterministic Timing Model. To meet the timing requirements at the same target clock frequency as the FluxArch Core, the RTL operators are partitioned into different numbers of pipeline stages. Table 2(b) lists the fixed RTL operator latencies using the same symbols as Fig. 9. Under the computation-only boundary used by the Engine model—excluding launch, stage-transition, `out_valid`, and CPU/MMIO input-wait cycles—the five stages take 10 cycles for **1** ($+ \rightarrow 1/x \rightarrow \times, 3 + 4 + 3$), 12 cycles for **2** and **3** combined ($+ \rightarrow \times \rightarrow + \rightarrow \times, 3 + 3 + 3 + 3$), 7 cycles for **4** ($1/x \rightarrow \times, 4 + 3$), and $3 + 4N_C$ cycles for **5** ($U \rightarrow \{+ \rightarrow <\}^{N_C}, 3 + N_C(3 + 1)$). The resulting latency is

$$L_{\text{Engine}}(N_N, N_C) = N_N(10 + 12N_C) + \max(7, 3 + 4N_C) \\ = 12N_N N_C + 10N_N + 4N_C + 3.$$

It depends only on the programmed nuclide and reaction-channel counts, not on the numerical operand values. The gem5 Engine timing directly implements this RTL-derived expression rather than an empirically fitted latency.

The operator latencies and five-stage timing parameters supporting this derivation are added in Table 2(b).

Added (Table 2(b), page 10):

Added

Table 2. Software-visible interface and deterministic RTL timing parameters of the FluxArch Engine: (a) MMIO register map; (b) fixed operator latencies in cycles.

	Offset	Register	Access	Function
	0x00	Control/Status	R/W	N_C [19:16], N_N [15:8], status; an accepted write launches execution
	0x08	Nuclide density	W	FP64 ρ_n , rewritten once per nuclide
(a)	0x10	Data Input A	W	Stage-reused FP64 input: E_1 , then E' , then σ_1
	0x18	Data Input B	W	Stage-reused FP64 input: E_0 , then σ_0
	0x20	Sampled distance	R	FP64 output d , valid when <code>out_valid</code> is asserted
	0x28	Reaction channel	R	Output c in [3:0], with [63:4] zero, valid when <code>out_valid</code> is asserted

	Operator	Cycles
(b)	+	3
	$\times$	3
	Exp	3
	U	3
	<	1
	$1/x$	4

Reviewer 2-Q1

The matched silicon/simulation calibration protocol, errors, and validation scope are described in **Section 6.1**.

Added (Section 6.1, page 13):

Core Timing Calibration. We calibrate the 2-GHz in-order FluxArch Core model against a 2-GB KickPi K3B with a 22-nm quad-core RK3562 Cortex-A53 [50, 55]. The native process is pinned to one 2-GHz core under stable thermal conditions; both platforms use identical source, GCC 13.3, `-O2 -fno-tree-vectorize`, seed, warm-up, and timed region. All six checksums match. Table 3 reports latency and signed error $100(T_{\text{gem5}} - T_{\text{K3B}})/T_{\text{K3B}}$; across the six tests, the mean and maximum absolute percentage errors are 6.7% and 13.5%, respectively. FluxArch’s absolute throughput and speedups are obtained directly from the calibrated simulations.

This silicon comparison calibrates the core and cache/DRAM paths, not the complete 1,024-core hierarchy. Section 4.2 establishes RTL-to-gem5 Engine timing traceability, and Sections 6.5–6.7 evaluate the modeled full system; we therefore report calibrated simulation evidence, not fabricated-system measurements.

References cited in this excerpt:

- [50] Rockchip Electronics Co., Ltd. [n. d.]. RK3562 Product Information. Rockchip product catalog. <https://www.rock-chips.com/a/en/products/> Accessed: 2026-08-23.
- [55] Shenzhen KICKPI Technology Co., Ltd. [n. d.]. KICKPI K3B Development Board. Official product page. <https://www.kickpi.com/product/k3b/> Accessed: 2026-08-28.

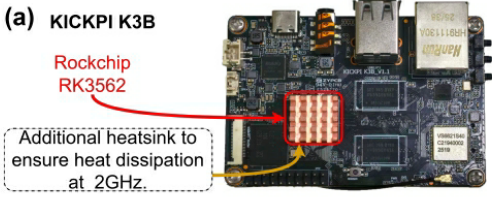
The six RK3562 calibration cases and measured-versus-simulated latencies are added in **Table 3**.

Added (Table 3, page 13):

Added

Table 3. Core timing calibration against the RK3562 Cortex-A53: (a) KickPi K3B board used as the silicon reference; (b) measured-versus-modeled latency.

(a) KICKPI K3B



Microbenchmark	K3B ns/operation	gem5 ns/operation	Error (%)
Integer, eight chains	6.18	6.52	+5.5
Branch, predictable	6.05	6.00	−0.8
Branch, pseudorandom	6.33	5.48	−13.5
Dependent load, 16 KiB	2.15	2.04	−5.0
Dependent load, 256 KiB	8.67	8.92	+2.8
Dependent load, 64 MiB	111.82	98.00	−12.4

Reviewer 2-Q1

2.2 Question 2

Resolve the L2-residency contradiction. Quantify per benchmark whether the replicated database fits in the 4 MB L2 (esp. XSBench); this conflicts with Fig. 3(e) and underpins the speedups.

Our performance evaluation does not assume that the benchmark databases reside entirely in L2. The gem5 cache model accounts for finite cache capacity, cache-line replacement, and accesses to the memory hierarchy following L2 misses. The reported throughput therefore reflects capacity-limited caching and the effects of the configured replacement policy, rather than idealized all-hit behavior. For XSBench, the 64.58% L2 read hit rate and 617.79 bytes of DRAM reads per lookup in Table 6(a) explicitly demonstrate that substantial off-chip accesses remain.

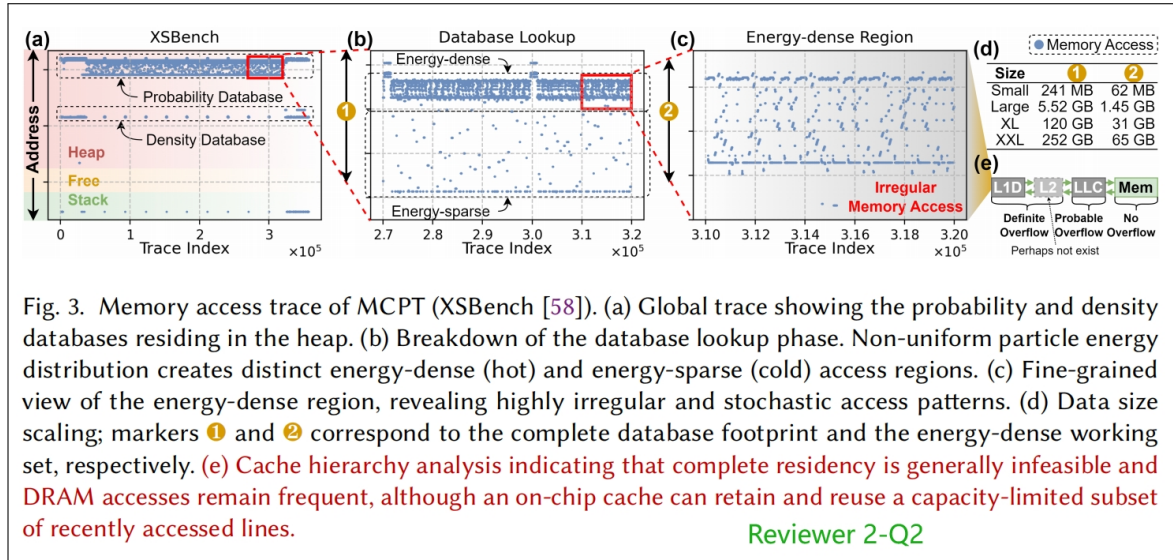
The capacity evidence in Fig. 3(d,e) is consistent with this evaluation. Fig. 3(d) lists a 241 MB complete database and a 62 MB energy-dense working set even for the Small configuration, both exceeding the 4 MB cluster-local L2. These sizes explain the cache-overflow schematic in Fig. 3(e): neither the complete database nor the energy-dense working set can fit in L2. However, capacity overflow does not preclude caching and reusing a subset of lines.

We revised the description of LLC-aware mapping to clarify that the strategy benefits from partial data reuse within each cluster’s shared L2 and does not require the entire benchmark database to fit in

its 4 MiB capacity. To support this clarification, we added per-workload database size, L2 hit rate, and normalized DRAM traffic. The Quicksilver databases occupy only 0.055–0.318 MiB, whereas the 241-MiB XSBench database is approximately $60.25\times$ the L2 capacity; XSBench therefore cannot reside in L2, although its shared read-only accesses still obtain partial temporal reuse.

The full-residency implication is corrected in **Section 5.2** and **Figure 3**, which now describes capacity-limited reuse within the shared L2.

Figure 3 (page 5):



Reviewer 2-Q2

Original (Section 5.2, page 12):

Locality-Optimized Data Replication: The hybrid model treats each process as an isolated memory domain, allowing us to replicate read-only density database and other shared data into local memory. Provided the dataset fits within the LLC, this converts memory-bound irregular lookups into low-latency L2 cache hits, bypassing the DRAM bottleneck.

Revised (Section 5.2, pages 11–12):

Shared-Database Reuse within an LLC Domain: Threads belonging to the same MPI process access a common read-mostly physics database. Binding these threads to cores sharing the cluster-local 4 MB L2 allows a cache line fetched by one thread to be reused by other threads in the process. This optimization does not require the complete database or working set to reside in L2. Instead, the shared L2 retains a capacity-limited subset of recently accessed cache lines, reducing rather than eliminating DRAM accesses. Each process places its database replica in its local NUMA memory domain, avoiding remote-memory accesses when an L2 miss occurs.

The workload-dependent L2 residency and DRAM traffic are analyzed in **Section 6.5**.

Added (Section 6.5, pages 17–18):

Table 6(a) clarifies that the affinity strategy does not rely on universal database residency in the 4 MB cluster-local L2. The databases of CORAL2-P1, CORAL2-P2, CTS2 and Homogeneous occupy only 0.055–0.318 MiB per process and achieve 91.98%–95.81% L2 read hit rates. In contrast, the 241 MiB XSBench database is approximately $60\times$ the L2 capacity. Nevertheless, its 64.58% L2 read hit rate shows that the cache retains a useful capacity-limited subset, while the remaining misses generate 617.79 bytes of DRAM reads per lookup. For Quicksilver, DRAM traffic is normalized per transported segment; for XSBench, it is normalized per cross-section lookup. These measurements are consistent with the intended mechanism: the shared L2 captures partial reuse but neither requires complete working-set residency nor eliminates DRAM traffic.

The database sizes, L2 hit rates, and normalized DRAM traffic are added in **Table 6(a)**.

Added (Table 6(a), page 18):

Table 6. Memory behavior, rank-level load balance, and inter-tile communication on the 1,024-core FluxArch system. Each MPI rank is bound to a 16-core cluster with a 4 MB shared L2. (a) Database footprint and memory behavior under LLC-aware mapping. (b) Rank-level work distribution and computation balance efficiency. (c) Peak directional TII utilization and particle-exchange time as a fraction of cycle-tracking time.

Added

Workload	(a)			(b)		(c)	
	DB/process (MiB)	L2 read hit rate	DRAM read (B/work)	Segment CV	Balance efficiency	Peak TII utilization (%)	Exchange Time
CORAL2-P1	0.318	92.07%	84.59	0.21%	97.72%	12.22	31.80%
CORAL2-P2	0.055	95.81%	140.81	0.16%	96.78%	28.15	26.59%
CTS2	0.318	91.98%	45.97	2.15%	95.45%	10.88	16.93%
Homogeneous	0.160	94.73%	200.82	5.53%	96.33%	19.73	30.00%
XSbench	241	64.58%	617.79	—	—	—	—

—: not reported or not applicable.

Reviewer 2-Q2

2.3 Question 3

Report DRAM-inclusive energy, or justify why excluding off-chip DRAM does not bias a memory-bound workload.

We agree that excluding DRAM would bias the system-level energy comparison for these memory-intensive workloads. We changed the evaluation to include co-packaged DDR4/HBM through device telemetry and off-package DDR5 through the same traffic-based DRAMPower methodology, and we vary the modeled DDR5 component over a $0.5\times-2\times$ range to assess sensitivity. The conservative process-scaled geometric-mean DRAM-inclusive energy-efficiency advantage remains $1.77\times$ over H100 and $6.65\times$ over Intel.

We now explain the choice of the $0.5\times-2\times$ range in **Section 6.1**. Ghose et al. [19] measured DDR3L read currents averaging 54.1% below datasheet values for one vendor and data-pattern-dependent read-power differences of up to 91.6%. Shi et al. [56] report that DDR4 calibration reduced DRAMPower energy estimates by roughly half for most evaluated SPEC2006 workloads. These results motivate testing sizable modeling discrepancies, but do not establish an error bound for our DDR5 configuration. We therefore select reciprocal factors of 0.5 and 2 to test both overestimation and underestimation, with package power fixed; the range is a sensitivity-analysis choice, not a statistical confidence interval.

For process-scaled energy-efficiency bounds, we independently vary only the scaled FluxArch package power by $\pm 30\%$ and each modeled off-package DRAM component over $0.5\times-2\times$, while keeping throughput and measured baseline package power fixed. We recompute the performance/W ratio at every endpoint combination and report its minimum and maximum; external DRAM power is not subject to process-node scaling or its 30% margin.

References cited in this excerpt:

- [19] Saugata Ghose, Abdullah Giray Yağlıkçı, Raghav Gupta, Donghyuk Lee, Kais Kudrolli, William X. Liu, Hasan Hassan, Kevin K. Chang, Niladrish Chatterjee, Aditya Agrawal, Mike O'Connor, and Onur Mutlu. 2018. What Your DRAM Power Models Are Not Telling You: Lessons from a Detailed Experimental Study. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 2 (2018), 41 pages. doi:10.1145/3224419
- [56] Xinyu Shi, Dina Ali Abdelhamid, Thomas Ilsche, Saeideh Alinezhad Chamazcoti, Timon Evenblij, Mohit Gupta, and Francky Catthoor. 2025. Calibrating DRAMPower Model for HPC: A Runtime Perspective from Real-Time Measurements. *arXiv:2411.17960v3*. <https://arxiv.org/abs/2411.17960v3> Revised September 3, 2025.

The common DRAM-power accounting model and sensitivity bounds are specified in **Section 6.1**.

The DRAM-inclusive energy-efficiency results and conservative lower bounds are updated in **Section 6.2**.

Original (Section 6.2, pages 14–15):

Iso-Process Energy Efficiency. As shown in Fig. 13(b), the energy efficiency advantage is even more pronounced. By comparing projected iso-process nodes, we isolate the architectural contributions. According to Section. 6.1, the power projection error is 30%. Therefore, we incorporate

this margin into our energy efficiency evaluation, which is represented as error bars in our results. Without the workload orchestration, the energy efficiency of FluxArch, mainstream server CPUs and high-end GPUs is at odds with each other. With the orchestration enabled, optimizing data locality and minimizing cross-tile traffic, FluxArch demonstrates a superior geometric mean energy efficiency improvement from $3.98\times$ (vs. FT 64-core, $5.69\times$ discounted by 30%) to $9.84\times$ (vs. Gold 6430 $\times 2$, $14.06\times$ discounted by 30%) even accounting for the potential projection error of pessimistic assumption. We also report the energy efficiency of the raw, unscaled data based on the 22 nm technology node. Even in this case, FluxArch still achieves an energy efficiency improvement of $1.17\times$ (vs. NVIDIA H100) to $4.59\times$ (vs. Gold 6430 $\times 2$).

Revised (Section 6.2, pages 15–16):

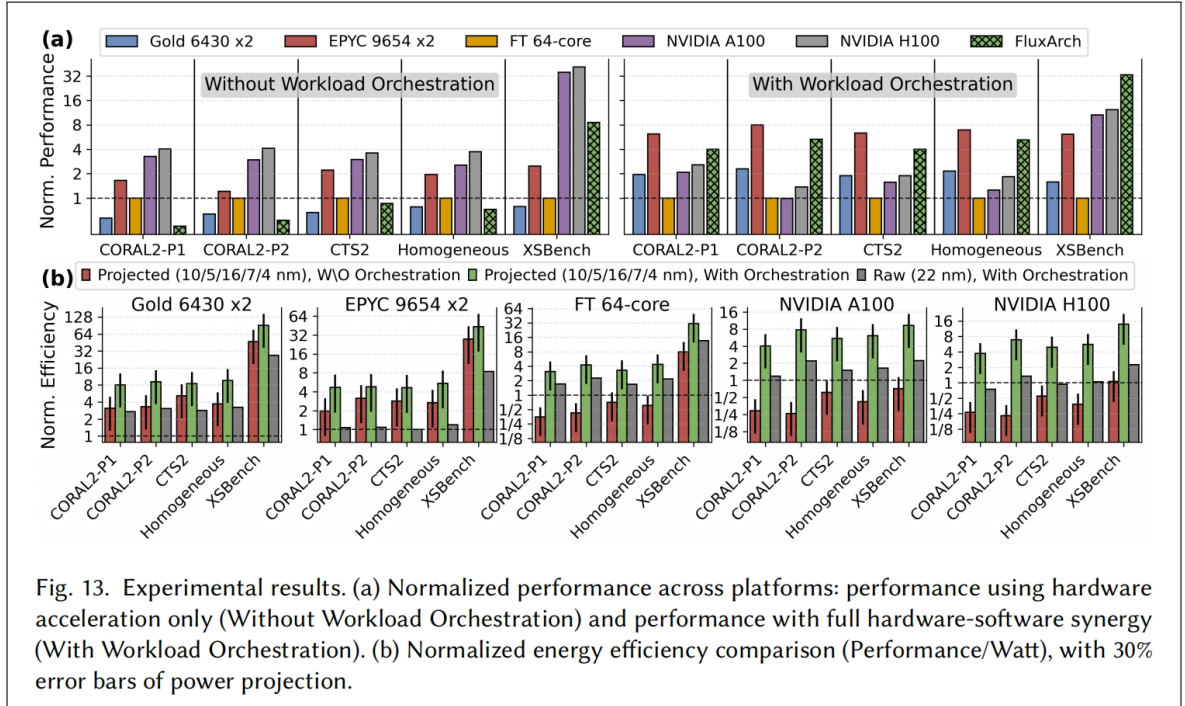
DRAM-Inclusive Energy Efficiency. Fig. 13(c,d) reports DRAM-inclusive power and energy efficiency under the sensitivity bounds defined in Section 6.1. Process-scaled results separate architectural effects from process scaling while accounting consistently for external memory.

In the process-scaled comparison, FluxArch improves geometric-mean DRAM-inclusive efficiency by $2.85\times$ over NVIDIA H100 to $10.58\times$ over Gold 6430 $\times 2$.

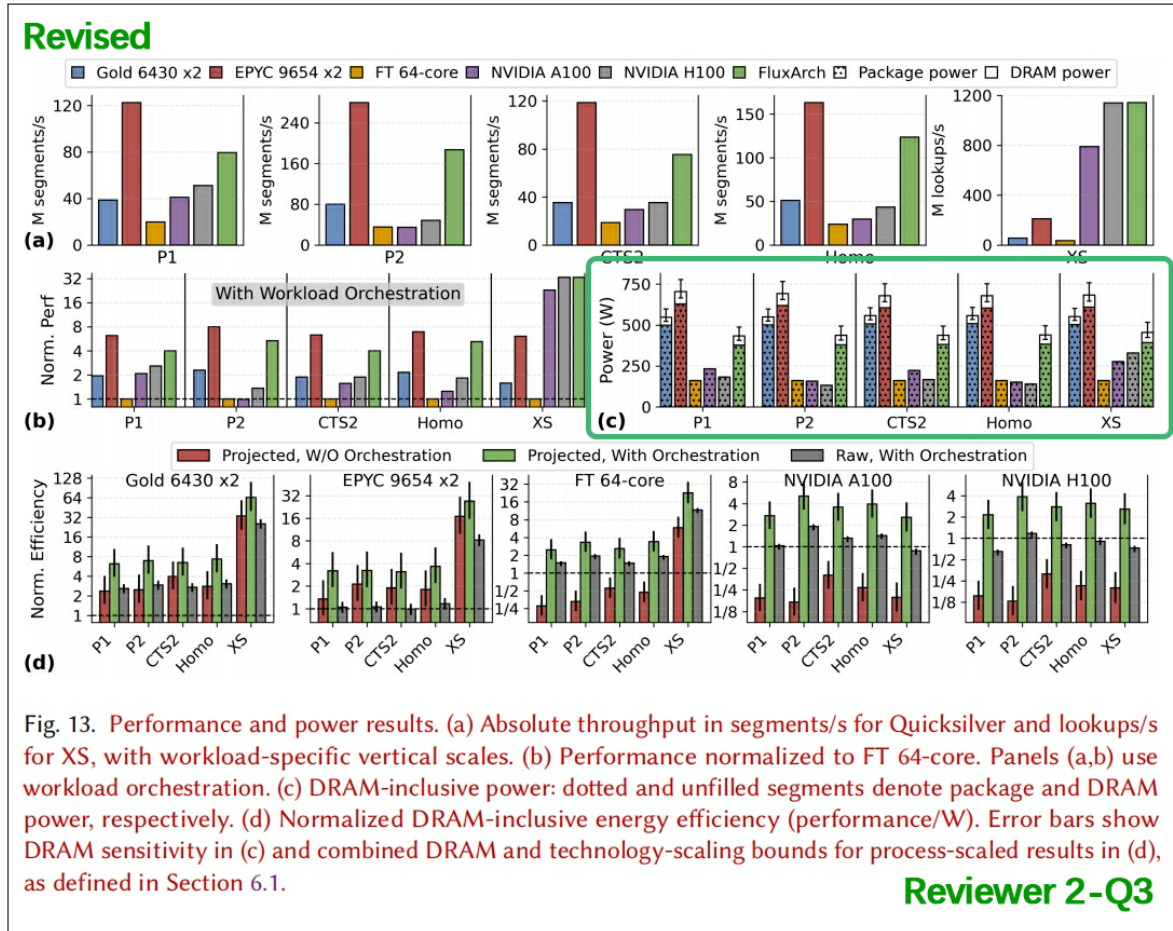
The sensitivity analysis preserves the process-scaled conclusion: across the combined DRAM and technology-scaling bounds, the geometric-mean improvement with orchestration remains $1.77\times$ over NVIDIA H100 and $6.65\times$ over Gold 6430 $\times 2$ at the conservative lower bound.

The DRAM-inclusive power breakdown and energy-efficiency results are plotted in revised **Figure 13(c,d)**. Panel (c) is newly added; panel (d) revises the original energy-efficiency plot.

Original (Figure 13, page 15):



Revised (Figure 13, page 16):



2.4 Question 4

Fix inconsistencies and add absolutes. Reconcile the conflicting engine-area and energy figures, report absolute throughput (histories/sec), and do a language pass.

We reconciled the Engine-area and energy-efficiency figures, added absolute throughput, and revised the language.

- (1) **Engine-area consistency.** The correct synthesized Engine area is 0.270 mm^2 , compared with 0.763 mm^2 for the standard FPU. We corrected the inconsistent value and percentage throughout the manuscript. The consistent result is that the Engine occupies 35.4% of the FPU area, corresponding to a 64.6% reduction.

The Engine-area value and its comparison with the standard FPU are corrected in **Section 3.3**.

Original (Section 3.3, page 7):

To evaluate the area efficiency of architectural specialization, we synthesized both a standard full-functional FPU and our engine using open-source RTL at a 22 nm technology node (as detailed in Section 6.3). By eliminating the extensive logic required for full FP32/FP64 precision and stripping away general-purpose scheduling, our dedicated engine occupies only 0.251 mm^2 —a 67% reduction in footprint compared to a standard FPU (0.763 mm^2). This specialized FP64-only design maximizes computational density per unit area, directly addressing the low arithmetic intensity of general-purpose cores while providing the throughput necessary for large-scale MCPT simulations.

Revised (Section 3.3, page 6):

To evaluate the area efficiency of architectural specialization, we synthesized both a standard full-functional FPU and our engine using open-source RTL [25] at a 22 nm technology node (as detailed in Section 6.3). By eliminating the extensive logic required for full FP32/FP64

precision and stripping away general-purpose scheduling, our dedicated engine occupies 0.270 mm^2 —a 64.6% reduction in footprint compared with a standard FPU (0.763 mm^2). This specialized FP64-only design maximizes computational density per unit area, directly addressing the low arithmetic intensity of general-purpose cores while providing the throughput necessary for large-scale MCPT simulations.

References cited in this excerpt:

- [25] John Hauser. 2024. *The HardFloat, SoftFloat, and TestFloat Packages*. <https://www.jhauser.us/arithmetic/>. Accessed: 2026-1-6.

The area analysis is corrected in **Section 6.3** to use the same Engine area and reduction percentage.

Original (Section 6.3, page 15):

The system hierarchy is optimized for high computational density (Fig. 14(c)), with Compute Tiles dominating the design (97% of total area) and compute cores accounting for approximately 70% of the entire system’s area. The FluxArch Engine, shown in Fig. 14(b), is remarkably area-efficient, occupying only one-third of the footprint of a standard FPU. This specialization for MCPT achieves higher throughput per mm^2 than general-purpose units. A detailed floorplan of the Engine (Fig. 14(a)) reveals a total area of 0.27 mm^2 , where FP64 functional units occupy over 97% of the engine’s area. This extreme functional density is achieved by stripping away single-precision support and complex control logic, dedicating nearly all silicon to the primary computational bottlenecks of MCPT.

Revised (Section 6.3, page 17):

The system hierarchy is optimized for high computational density (Fig. 14(c)), with Compute Tiles dominating the design (97% of total area) and compute cores accounting for approximately 70% of the entire system’s area. **The FluxArch Engine, shown in Fig. 14(b), is remarkably area-efficient, occupying 0.270 mm^2 , or 35.4% of the 0.763 mm^2 standard-FPU footprint.** This specialization for MCPT achieves higher throughput per mm^2 than general-purpose units. **A detailed floorplan of the Engine (Fig. 14(a)) shows that FP64 functional units occupy over 97% of the engine area.** This extreme functional density is achieved by stripping away single-precision support and complex control logic, dedicating nearly all silicon to the primary computational bottlenecks of MCPT.

- (2) **Energy-efficiency consistency.** The original manuscript mixed nominal energy-efficiency improvements ($5.69\times$ – $14.06\times$ in the Introduction) with conservative values after a 30% discount ($3.98\times$ – $9.84\times$ in Section 6.2 and the Conclusion), without consistently distinguishing the two. We have unified the reporting convention for the updated evaluation: the Abstract, Introduction, Section 6.2, and Conclusion now report nominal process-scaled, DRAM-inclusive energy-efficiency results, with geometric-mean improvements of $2.85\times$ over H100 to $10.58\times$ over Intel (the Abstract reports the maximum geometric-mean improvement across the evaluated baselines). Section 6.2 separately identifies $1.77\times$ over H100 and $6.65\times$ over Intel as conservative lower bounds under the combined DRAM and technology-scaling uncertainties. These updated results incorporate the revised DRAM accounting and GPU baseline described in our other responses; they are not a relabeling of the original measurements.

The evaluation contribution in **Section 1** is revised to report the updated nominal process-scaled, DRAM-inclusive energy-efficiency range.

Original (Section 1, page 3):

We evaluate FluxArch via full-system modeling across industrial-grade benchmarks. Experimental results demonstrate that FluxArch achieves a geometric mean speedup of $1.02\times$ to $6.83\times$ and energy efficiency improvements of $5.69\times$ to $14.06\times$ compared to high-performance server-class CPUs and GPUs.

Revised (Section 1, page 2):

We evaluate FluxArch via full-system modeling across industrial-grade benchmarks. The evaluation combines package power with measured or traffic-modeled memory power, depending on whether memory is co-packaged or off-package, and uses a sorting-enabled XSBench implementation as the GPU baseline. **Experimental results demonstrate that FluxArch achieves a geometric mean speedup of $1.02\times$ to $6.83\times$ and process-scaled DRAM-inclusive energy-efficiency improvements of $2.85\times$ to $10.58\times$ compared to high-performance server-class**

CPUs and GPUs.

The result summary in **Section 9** is revised to replace the original discounted energy-efficiency range with the updated nominal process-scaled, DRAM-inclusive range, consistent with Section 1.

Original (Section 9, page 20):

By co-optimizing hardware topology and spatial decomposition, FluxArch overcomes irregular memory access limitations and delivers geometric mean speedups of up to $1.02\times$ – $6.83\times$ and energy efficiency improvements of up to $3.98\times$ – $9.84\times$ over traditional processors.

Revised (Section 9, page 23):

By co-optimizing hardware topology and spatial decomposition, FluxArch overcomes irregular memory access limitations. Using a sorting-enabled XSBench implementation for the GPU comparison, FluxArch delivers geometric mean speedups of $1.02\times$ – $6.83\times$ over traditional processors. **It also delivers process-scaled DRAM-inclusive energy-efficiency improvements of $2.85\times$ – $10.58\times$.**

The nominal results and conservative lower bounds are distinguished explicitly in **Section 6.2**.

Revised (Section 6.2, page 16):

In the process-scaled comparison, FluxArch improves geometric-mean DRAM-inclusive efficiency by $2.85\times$ over NVIDIA H100 to $10.58\times$ over Gold 6430 $\times 2$. The sensitivity analysis preserves the process-scaled conclusion: across the combined DRAM and technology-scaling bounds, the geometric-mean improvement with orchestration remains $1.77\times$ over NVIDIA H100 and $6.65\times$ over Gold 6430 $\times 2$ at the conservative lower bound.

- (3) **Absolute throughput.** The original normalized plot did not expose the absolute performance represented by each workload-specific metric. We revised the evaluation to report absolute throughput alongside the normalized comparison, using segments/s for CORAL2-P1, CORAL2-P2, CTS2 and Homogeneous and lookups/s for XSBench. We report the benchmark-specific equivalents of histories/s: segments/s for Quicksilver and lookups/s for XSBench. FluxArch achieves 79.33, 186.70, 75.33, and 123.50 million segments/s for P1, P2, CTS2, and Homogeneous, respectively, and 1.143 billion XSBench lookups/s.

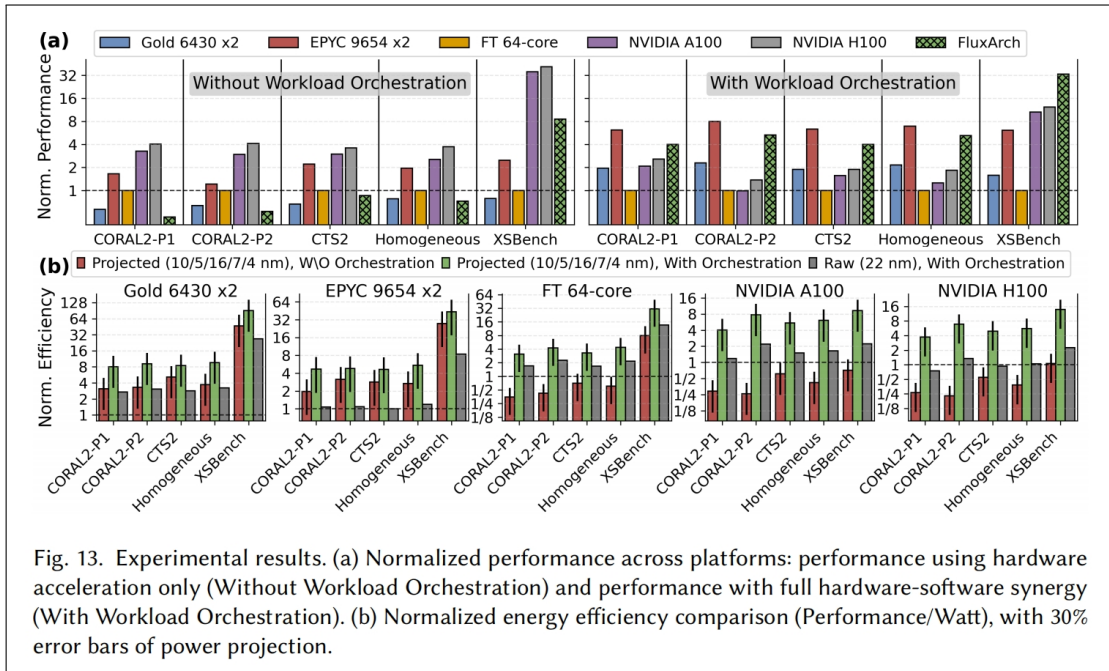
The absolute throughput values and workload-specific units are reported in **Section 6.2**.

Revised (Section 6.2, page 15):

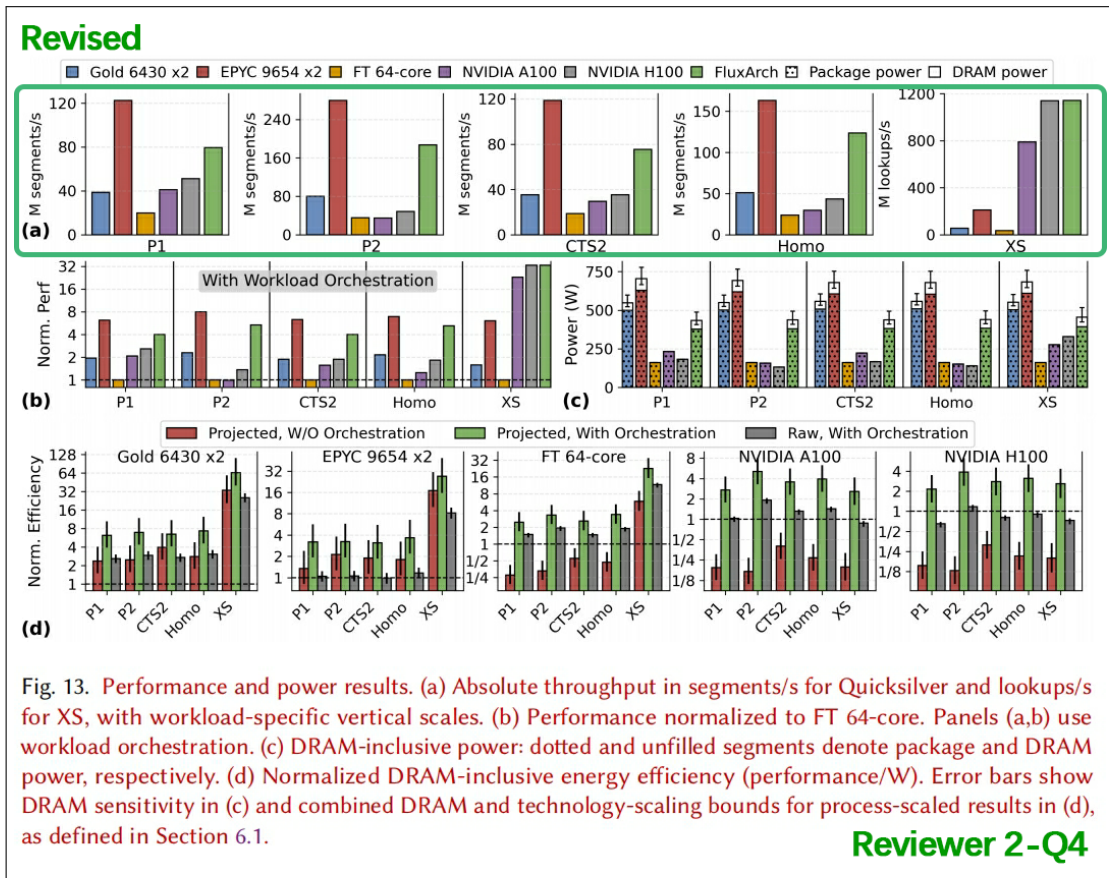
Performance Impact. Fig. 13(a,b) reports absolute throughput and values normalized to FT 64-core. FluxArch achieves 79.33, 186.70, 75.33, and 123.50 million segments/s for P1, P2, CTS2, and Homo, respectively, and 1.143 billion XS lookups/s, yielding geometric-mean speedups from **$1.02\times$** over EPYC 9654 $\times 2$ to **$6.83\times$** over FT 64-core.

Figure 13(a) adds absolute throughput, while **Figure 13(b)** revises the original normalized-performance plot.

Original (Figure 13, page 15):



Revised (Figure 13, page 16):

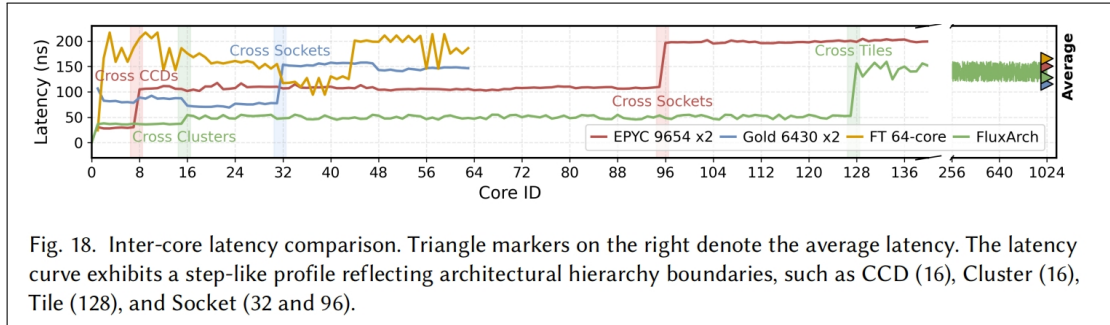


Reviewer 2-Q4

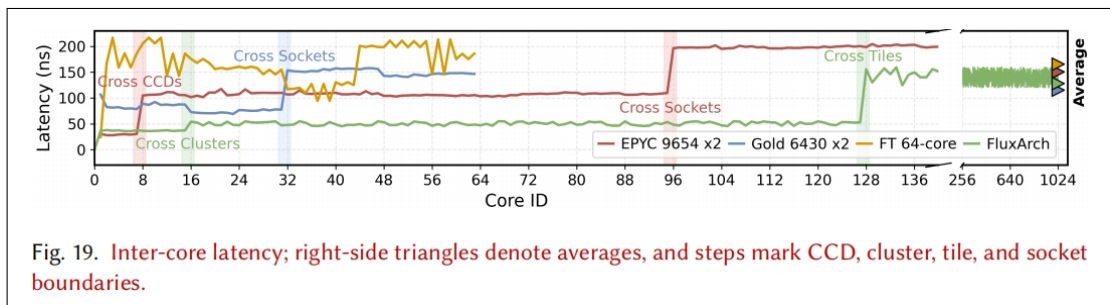
- (4) **Language revision.** We carefully reviewed the language throughout the manuscript. Specifically, we corrected spelling and typographical errors, fixed grammatical issues, revised awkward sentence structures, and rephrased passages to improve clarity, fluency, and readability. For example, in the caption of **Figure 19** in **Section 6.7**, we condensed the original three-sentence description into: “Inter-core latency; right-side triangles denote averages, and steps mark CCD,

cluster, tile, and socket boundaries.” This replaces verbose wording such as “The latency curve exhibits a step-like profile reflecting architectural hierarchy boundaries” with the more direct “steps mark CCD, cluster, tile, and socket boundaries,” while retaining the explanation of the plotted markers and hierarchy boundaries.

Original (Figure 18, page 18):



Revised (Figure 19, page 21):



2.5 Question 5

Acknowledged but not quantified: DRAM-inclusive energy (Sec. 6.1 excludes it); whether the working set fits in 4 MB L2 — which contradicts Fig. 3(e); load imbalance on non-uniform inputs (Sec. 7). Absent: inter-tile MPI transport and Death-Respawn cost; the MMIO programming/porting interface; absolute throughput; model validation against silicon.

We sincerely thank you for your helpful and constructive suggestion. Because this comment collects eight distinct issues, we address them separately.

- (1) **DRAM-inclusive energy.** The energy comparison now includes main-memory power rather than using an on-chip-only boundary. We added consistent accounting for co-packaged memory and off-package DDR5 plus a $0.5\times-2\times$ sensitivity analysis. The conservative process-scaled advantage remains above unity.

The memory-power accounting boundary and DRAM sensitivity analysis are clarified in **Section 6.1**.

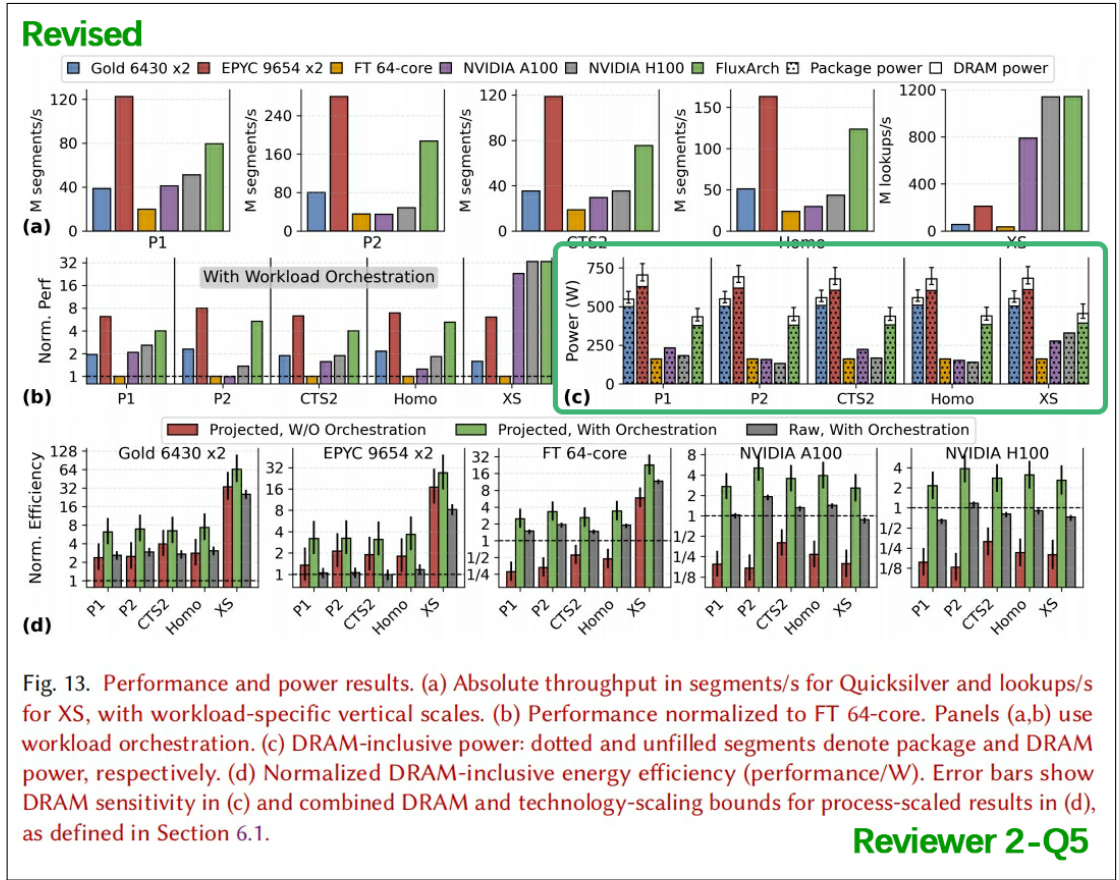
Revised (Section 6.1, page 14):

We report DRAM-inclusive system power and energy. Package power includes cores, caches, NoCs, and integrated memory controllers; for FT 64-core, NVIDIA A100, and NVIDIA H100, the main memory is co-packaged (DDR4 or HBM) and is consequently covered by platform-level package/device telemetry. In contrast, the DDR5 DIMMs of Gold 6430 $\times 2$ and EPYC 9654 $\times 2$, and the DDR5 channels assumed for FluxArch, are physically off-package and are not covered by package-power readings. We therefore add a separately modeled DRAM component to these platforms, avoiding asymmetric accounting that would include HBM/DDR4 power for the co-packaged-memory platforms but omit DDR5 power for the other platforms.

The resulting energy-efficiency comparison is updated in **Section 6.2**.

The DRAM-inclusive power and energy-efficiency plots are updated in **Figure 13(c,d)**. Panel (c) is newly added; panel (d) revises the original energy-efficiency plot.

Revised (Figure 13, page 16):



- (2) **L2 working-set residency.** Our affinity mechanism provides partial shared-L2 reuse rather than universal full-working-set residency. We corrected the claim and added database-capacity and memory-traffic measurements. The 241-MiB XSBench database is about $60.25\times$ larger than the 4-MiB L2, confirming that its benefit cannot be attributed to complete residency. The affinity mechanism is clarified in **Section 5.2** as partial shared-L2 reuse rather than full database residency.

Revised (Section 5.2, pages 11–12):

Shared-Database Reuse within an LLC Domain: Threads belonging to the same MPI process access a common read-mostly physics database. Binding these threads to cores sharing the cluster-local 4 MB L2 allows a cache line fetched by one thread to be reused by other threads in the process. This optimization does not require the complete database or working set to reside in L2. Instead, the shared L2 retains a capacity-limited subset of recently accessed cache lines, reducing rather than eliminating DRAM accesses. Each process places its database replica in its local NUMA memory domain, avoiding remote-memory accesses when an L2 miss occurs.

The memory-behavior analysis is added in **Section 6.5**.

The supporting database-capacity, L2-hit-rate, and DRAM-traffic measurements are added in **Table 6(a)**.

Added (Table 6(a), page 18):

Table 6. Memory behavior, rank-level load balance, and inter-tile communication on the 1,024-core FluxArch system. Each MPI rank is bound to a 16-core cluster with a 4 MB shared L2. (a) Database footprint and memory behavior under LLC-aware mapping. (b) Rank-level work distribution and computation balance efficiency. (c) Peak directional TII utilization and particle-exchange time as a fraction of cycle-tracking time.

Added

Workload	(a)			(b)		(c)	
	DB/process (MiB)	L2 read hit rate	DRAM read (B/work)	Segment CV	Balance efficiency	Peak TII utilization (%)	Exchange Time
CORAL2-P1	0.318	92.07%	84.59	0.21%	97.72%	12.22	31.80%
CORAL2-P2	0.055	95.81%	140.81	0.16%	96.78%	28.15	26.59%
CTS2	0.318	91.98%	45.97	2.15%	95.45%	10.88	16.93%
Homogeneous	0.160	94.73%	200.82	5.53%	96.33%	19.73	30.00%
XSbench	241	64.58%	617.79	—	—	—	—

—: not reported or not applicable.

Reviewer 2-Q5

- (3) **Load imbalance.** Static spatial decomposition can create unequal stochastic work, so we added measurements across all 64 cluster-bound MPI ranks. Kernel-time CV is 1.17%–2.01%, balance efficiency is 95.45%–97.72%, and the estimated computation-side tail loss is at most 4.55% for the evaluated inputs. These results indicate that static affinity and spatial decomposition maintain balanced computation across clusters for the evaluated workloads at 1,024 cores, with limited computation-side loss from load imbalance. More extreme inputs remain a limitation. The rank-level load-balance evaluation and estimated computation-side tail loss are added in **Section 6.5**.

Added (Section 6.5, pages 18–19):

Load Balance. Table 6(b) quantifies the concern that static affinity and spatial decomposition may assign unequal work to clusters. We record completed segments and accumulated tracking-kernel time separately for all 64 ranks. Segment-count CV ranges from 0.16% to 5.53%; because stochastic segments can follow different paths and incur different costs, count variation does not translate proportionally into execution-time variation. Kernel-time CV remains 1.17%–2.01%. We define balance efficiency as the mean rank-level kernel time divided by the maximum rank-level kernel time, since the slowest rank determines the computation tail. The resulting efficiency is 95.45%–97.72%, corresponding to an estimated computation-side tail loss of at most 4.55%. These results indicate that static affinity and spatial decomposition maintain balanced computation across clusters for the evaluated workloads at 1,024 cores, with limited computation-side loss from load imbalance.

The segment-count variation and balance efficiency are reported in **Table 6(b)**.

Added (Table 6(b), page 18):

Table 6. Memory behavior, rank-level load balance, and inter-tile communication on the 1,024-core FluxArch system. Each MPI rank is bound to a 16-core cluster with a 4 MB shared L2. (a) Database footprint and memory behavior under LLC-aware mapping. (b) Rank-level work distribution and computation balance efficiency. (c) Peak directional TII utilization and particle-exchange time as a fraction of cycle-tracking time.

Added

Workload	(a)			(b)		(c)	
	DB/process (MiB)	L2 read hit rate	DRAM read (B/work)	Segment CV	Balance efficiency	Peak TII utilization (%)	Exchange Time
CORAL2-P1	0.318	92.07%	84.59	0.21%	97.72%	12.22	31.80%
CORAL2-P2	0.055	95.81%	140.81	0.16%	96.78%	28.15	26.59%
CTS2	0.318	91.98%	45.97	2.15%	95.45%	10.88	16.93%
Homogeneous	0.160	94.73%	200.82	5.53%	96.33%	19.73	30.00%
XSbench	241	64.58%	617.79	—	—	—	—

—: not reported or not applicable.

Reviewer 2-Q5

- (4) **Inter-tile MPI transport.** Inter-tile MPI transport is explicitly modeled rather than implemented as zero-cost functional communication. We added the complete path through the guest network stack, IGbE endpoints, and DistEtherLink TIIs and measured traffic at every interface.

The busiest direction uses 10.88%–28.15% of the modeled 1-Gb/s capacity, with only 0.50%–2.42% traffic CV across TIIs.

The guest-network, IGbE, and TII transport path is specified in **Section 6.1**.

Added (Section 6.1, page 13):

Inter-Tile Communication Modeling. The eight Compute Tiles are connected to the Switch Tile through eight bidirectional Tile Interconnection Interfaces (TIIs). In distributed full-system gem5, each TII is modeled using a **DistEtherLink** and an IGbE endpoint, with the directional 1-Gb/s capacity in Table 1 and the modeled inter-tile latency. MPI messages traverse the guest network stack and these modeled links and are therefore subject to software protocol processing, packetization, link serialization, and inter-tile latency rather than zero-cost functional communication. We use the Ethernet-device counters to measure transmitted bytes and directional utilization at every TII. Quicksilver’s cycle-tracking timers separately measure end-to-end particle-exchange time, including buffer handling, MPI communication and waiting, and receive-side particle management.

The measured inter-tile traffic and link-utilization results are analyzed in **Section 6.7**.

Added (Section 6.7, page 21):

Inter-Tile Communication Cost. Table 6(c) reports peak directional TII utilization and the particle-exchange share of cycle-tracking time. Across CORAL2-P1, CORAL2-P2, CTS2 and Homogeneous, FluxArch injects 1.37–1.46 bytes of inter-tile traffic per completed segment. The busiest directional TII reaches only 10.88%–28.15% of its modeled 1-Gb/s capacity. Bidirectional link traffic is also well balanced: its coefficient of variation across the eight TIIs is 0.50%–2.42%. Thus, the communication-driven spatial mapping avoids TII hotspots, and the evaluated links remain below saturation.

The peak directional TII utilization and particle-exchange share are reported in **Table 6(c)**.

Added (Table 6(c), page 18):

Table 6. Memory behavior, rank-level load balance, and inter-tile communication on the 1,024-core FluxArch system. Each MPI rank is bound to a 16-core cluster with a 4 MB shared L2. (a) Database footprint and memory behavior under LLC-aware mapping. (b) Rank-level work distribution and computation balance efficiency. (c) Peak directional TII utilization and particle-exchange time as a fraction of cycle-tracking time.

Added

Workload	(a)			(b)		(c)	
	DB/process (MiB)	L2 read hit rate	DRAM read (B/work)	Segment CV	Balance efficiency	Peak TII utilization (%)	Exchange Time
CORAL2-P1	0.318	92.07%	84.59	0.21%	97.72%	12.22	31.80%
CORAL2-P2	0.055	95.81%	140.81	0.16%	96.78%	28.15	26.59%
CTS2	0.318	91.98%	45.97	2.15%	95.45%	10.88	16.93%
Homogeneous	0.160	94.73%	200.82	5.53%	96.33%	19.73	30.00%
XSbench	241	64.58%	617.79	—	—	—	—

—: not reported or not applicable.

Reviewer 2-Q5

- (5) **Death-Respawn cost.** Death-Respawn and particle exchange are included in the evaluated execution path. We added an end-to-end timer covering send-buffer handling, MPI communication and waiting, receive-side particle management, and termination-related communication. This phase accounts for 16.93%–31.80% of cycle-tracking time, showing that it is material even though the modeled TIIs are not saturated.

The end-to-end particle-exchange measurement boundary and its distinction from pure link-transfer time are clarified in **Section 6.7**.

Added (Section 6.7, page 21):

Particle exchange nevertheless accounts for 16.93%–31.80% of cycle-tracking time. This end-to-end phase includes send-buffer handling, MPI communication and waiting, receive-side particle management, and termination-related communication; it should therefore not be interpreted as pure link-transfer time. Together, the measurements show that particle exchange has a measurable execution cost, but that cost cannot be attributed to saturation of the modeled TII bandwidth.

The particle-exchange share of cycle-tracking time is reported in **Table 6(c)**.

Added (Table 6(c), page 18):

Table 6. Memory behavior, rank-level load balance, and inter-tile communication on the 1,024-core FluxArch system. Each MPI rank is bound to a 16-core cluster with a 4 MB shared L2. (a) Database footprint and memory behavior under LLC-aware mapping. (b) Rank-level work distribution and computation balance efficiency. (c) Peak directional TII utilization and particle-exchange time as a fraction of cycle-tracking time.

Added

Workload	(a)			(b)		(c)	
	DB/process (MiB)	L2 read hit rate	DRAM read (B/work)	Segment CV	Balance efficiency	Peak TII utilization (%)	Exchange Time
CORAL2-P1	0.318	92.07%	84.59	0.21%	97.72%	12.22	31.80%
CORAL2-P2	0.055	95.81%	140.81	0.16%	96.78%	28.15	26.59%
CTS2	0.318	91.98%	45.97	2.15%	95.45%	10.88	16.93%
Homogeneous	0.160	94.73%	200.82	5.53%	96.33%	19.73	30.00%
XSbench	241	64.58%	617.79	—	—	—	—

—: not reported or not applicable.

Reviewer 2-Q5

- (6) **MMIO programming and porting interface.** Each Engine is directly attached to its core’s execute stage, as shown in **Figure 7**. Its MMIO interface addresses core-local registers, avoiding the interconnect traversal associated with remote MMIO peripherals. Ordinary loads and stores access the 64-bit register map, so the interface requires neither an ISA extension nor compiler modification. We clarified this local connection and added the complete operand order, launch sequence, completion protocol, and software-porting boundary in **Section 4.2** and added **Table 2(a)**. As described in **Section 6.4**, the evaluated execution path still includes the MMIO instructions, Engine synchronization, and Engine execution latency.

The core-local Engine connection, launch/completion protocol, and software-porting requirements are clarified in **Section 4.2**.

Revised (Section 4.2, page 10):

Control and Programming Interface. As shown in Fig. 7, each FluxArch Engine is directly attached to its core’s execute stage. Its memory-mapped interface provides software access to core-local registers, avoiding the interconnect traversal associated with remote MMIO peripherals. The Control Unit maintains the iteration bounds and drives the nested loops in the FSM. Through the 64-bit MMIO interface in Table 2(a), the host supplies the loop bounds; an accepted Control/Status write resets the internal counters and launches execution. The host then streams operands and retrieves the results after `out_valid` indicates completion.

The interface is ISA-agnostic: the CPU performs database lookup in software, writes the resulting operands using ordinary stores, and reads the two final results using ordinary loads. Porting therefore replaces only the stable software kernel with an MMIO wrapper; it requires neither ISA extensions nor changes to the compiler, particle representation, database layout, or MPI+OpenMP orchestration.

The operand order and software-visible register map are added in **Table 2(a)**.

Added (Table 2(a), page 10):

Added

Table 2. Software-visible interface and deterministic RTL timing parameters of the FluxArch Engine: (a) MMIO register map; (b) fixed operator latencies in cycles.

	Offset	Register	Access	Function	(b) Operator Cycles	
(a)	0x00	Control/Status	R/W	N_C [19:16], N_N [15:8], status; an accepted write launches execution	+	3
	0x08	Nuclide density	W	FP64 ρ_n , rewritten once per nuclide	×	3
	0x10	Data Input A	W	Stage-reused FP64 input: E_1 , then E' , then σ_1	Exp	3
	0x18	Data Input B	W	Stage-reused FP64 input: E_0 , then σ_0	U	3
	0x20	Sampled distance	R	FP64 output d , valid when <code>out_valid</code> is asserted	<	1
	0x28	Reaction channel	R	Output c in [3:0], with [63:4] zero, valid when <code>out_valid</code> is asserted	1/x	4

Reviewer 2-Q5

- (7) **Absolute throughput.** We supplemented normalized speedup with absolute throughput for every workload and platform. The revised results use segments/s for Quicksilver and lookups/s for XSbench and derive the normalized values from the same measurements. FluxArch reaches

75.33–186.70 million segments/s across the Quicksilver inputs and 1.143 billion XSBench lookups/s. The absolute throughput values and the units for Quicksilver and XSBench are specified in Section 6.2.

Original (Section 6.2, page 14):

Performance Impact. Fig. 13(a) presents the performance of platforms across five benchmarks, normalized to the FT 64-core. The results underscore a cooperation between the domain-specific architecture and the synergistic workload orchestration.

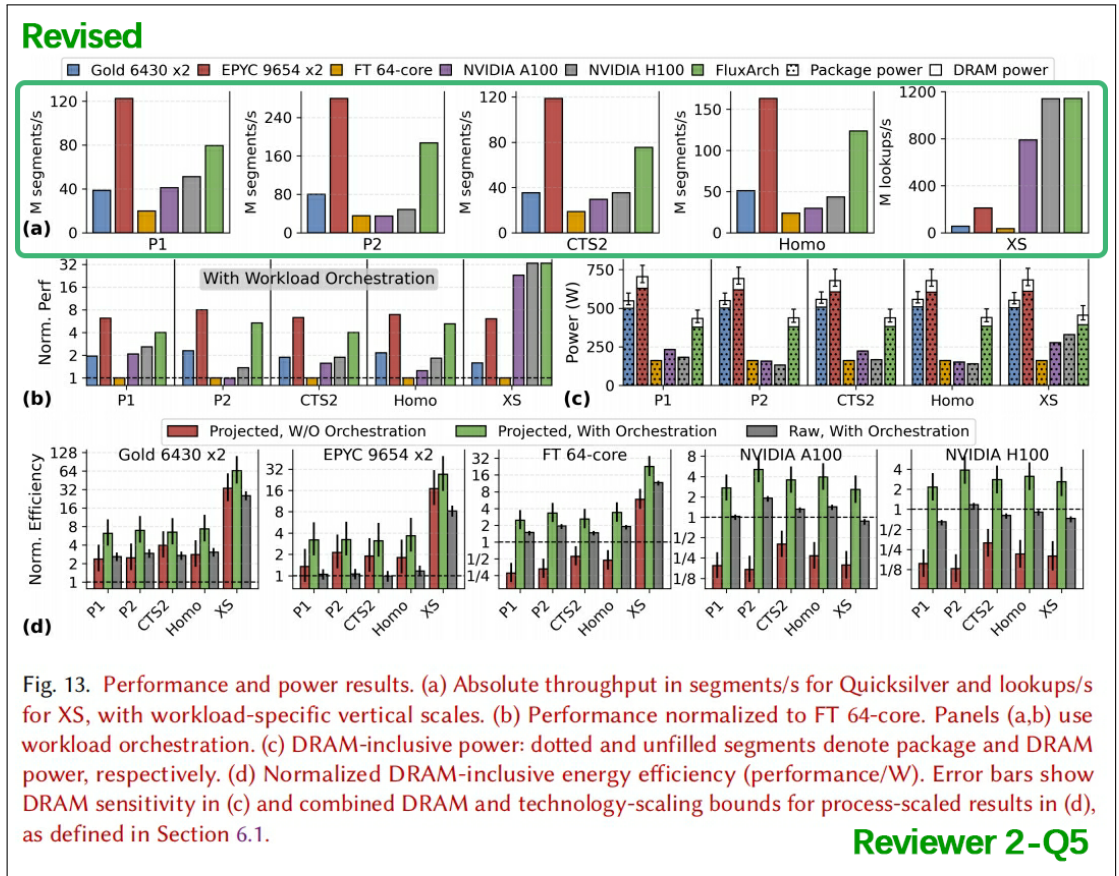
Without the synergistic workload orchestration, FluxArch offers little to no advantage over CPUs and GPUs, ranging from $0.17\times$ (vs. NVIDIA H100) to $1.54\times$ (vs. Gold 6430 $\times 2$), due to severe interconnect contention and non-local memory traffic. Crucially, enabling the workload orchestration unlocks the full potential of FluxArch to surpass all of the platforms, boosting the geometric mean speedup from $1.02\times$ (vs. EPYC 9654 $\times 2$) to $6.83\times$ (vs. FT 64-core).

Revised (Section 6.2, page 15):

Performance Impact. Fig. 13(a,b) reports absolute throughput and values normalized to FT 64-core. FluxArch achieves 79.33, 186.70, 75.33, and 123.50 million segments/s for P1, P2, CTS2, and Homo, respectively, and 1.143 billion XS lookups/s, yielding geometric-mean speedups from $1.02\times$ over EPYC 9654 $\times 2$ to $6.83\times$ over FT 64-core.

The per-platform absolute throughput and corresponding normalized results are presented in Figure 13(a,b). Panel (a) is newly added; panel (b) revises the original normalized-performance plot.

Revised (Figure 13, page 16):



- (8) **Model validation against silicon.** As detailed in our response to **R2-Q1, item (2)**, six RK3562 silicon comparisons calibrate the core and cache/DRAM paths, with 6.7% mean and 13.5% maximum absolute percentage errors. The RTL-derived Engine timing establishes implementation traceability, not independent silicon validation; the complete 1,024-core hierarchy remains evaluated through calibrated simulation rather than a fabricated prototype. The supporting manuscript excerpts and calibration results are provided in that response. The distinction between native baseline measurements and simulated FluxArch results is documented in **R2-Q1, item (1)**.

3 Reviewer 3

3.1 Question 1

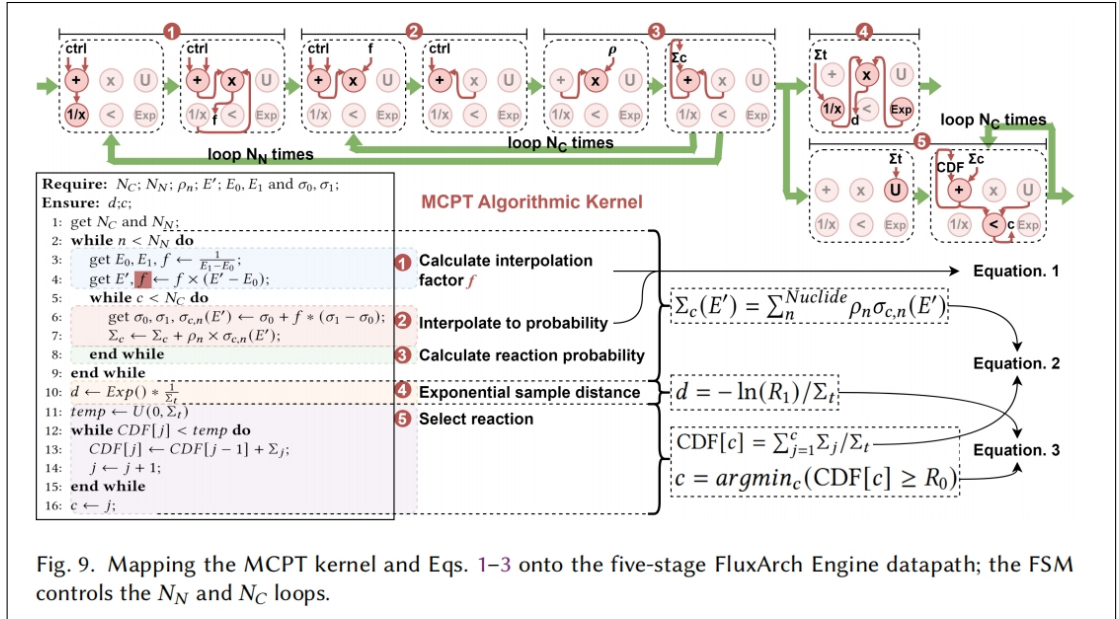
The performance model is unvalidated. This is the central weakness.

FluxArch is evaluated in gem5, with McPAT and CACTI for power and area. The cores derive from real ARM in-order cores, which helps. The full system does not. A 1,024-core design with a custom dataflow engine, a tree interconnect, and a NUMA chiplet hierarchy is never checked against silicon, RTL, or an existing many-core platform. The only error bar in the paper (30%) covers the technology-node power projection, not the timing model. Every performance and efficiency number depends on this uncalibrated model. The $6.83\times$ speedup cannot be assessed without validation evidence.

We sincerely thank you for your helpful and constructive suggestion. We agree that the performance claims require explicit validation evidence and a clear statement of what remains simulation-based. We therefore address the Engine, core/cache path, and complete system separately.

- (1) **RTL-derived Engine timing.** The Engine is a deterministic dataflow implementation whose latency depends on the nuclide count (N_N) and reaction-channel count per nuclide (N_C) rather than on operand values. We added an RTL-to-gem5 derivation using the fixed operator latencies and the dependencies among the five stages. This produces the closed-form computation latency $12N_N N_C + 10N_N + 4N_C + 3$, which gem5 implements directly instead of fitting an empirical delay.

The deterministic Engine timing expression and its RTL derivation are explained in **Section 4.2. Figure 9 (page 9)**:



Added (Section 4.2, pages 9–10):

Here, N_N denotes the number of nuclides and N_C the number of reaction channels per nuclide; they determine the outer and inner loop bounds, respectively.

Deterministic Timing Model. To meet the timing requirements at the same target clock frequency as the FluxArch Core, the RTL operators are partitioned into different numbers of pipeline stages. Table 2(b) lists the fixed RTL operator latencies using the same symbols as Fig. 9. Under the computation-only boundary used by the Engine model—excluding launch, stage-transition, `out_valid`, and CPU/MMIO input-wait cycles—the five stages take 10 cycles for ① ($+ \rightarrow 1/x \rightarrow \times, 3 + 4 + 3$), 12 cycles for ② and ③ combined ($+ \rightarrow \times \rightarrow + \rightarrow \times, 3 + 3 + 3 + 3$), 7 cycles for ④ ($1/x \rightarrow \times, 4 + 3$), and $3 + 4N_C$ cycles for ⑤ ($U \rightarrow \{+ \rightarrow <\}^{N_C}, 3 + N_C(3 + 1)$). The resulting latency is

$$\begin{aligned}
 L_{\text{Engine}}(N_N, N_C) &= N_N(10 + 12N_C) + \max(7, 3 + 4N_C) \\
 &= 12N_N N_C + 10N_N + 4N_C + 3.
 \end{aligned}$$

It depends only on the programmed nuclide and reaction-channel counts, not on the numerical operand values. The gem5 Engine timing directly implements this RTL-derived expression rather than an empirically fitted latency.

The operator latencies and five-stage timing parameters are specified in **Table 2(b)**.

Added (Table 2, page 10):

Added

Table 2. Software-visible interface and deterministic RTL timing parameters of the FluxArch Engine: (a) MMIO register map; (b) fixed operator latencies in cycles.

	Offset	Register	Access	Function
	0x00	Control/Status	R/W	N_C [19:16], N_N [15:8], status; an accepted write launches execution
	0x08	Nuclide density	W	FP64 ρ_n , rewritten once per nuclide
(a)	0x10	Data Input A	W	Stage-reused FP64 input: E_1 , then E' , then σ_1
	0x18	Data Input B	W	Stage-reused FP64 input: E_0 , then σ_0
	0x20	Sampled distance	R	FP64 output d , valid when out_valid is asserted
	0x28	Reaction channel	R	Output c in [3:0], with [63:4] zero, valid when out_valid is asserted

	Operator	Cycles
(b)	+	3
	$\times$	3
	Exp	3
	U	3
	<	1
	1/x	4

Reviewer 3-Q1

- (2) **Silicon-based core and memory calibration.** The in-order core and cache/DRAM paths are anchored to a Cortex-A53-class silicon reference. We now report the details of six matched native/simulated configurations spanning integer execution, branch control, and dependent random accesses from 16 KiB to 64 MiB. All checksums match, while the measured latency differences yield 6.7% mean absolute percentage error and 13.5% maximum absolute percentage error. The silicon-based core and memory calibration protocol, accuracy, and scope are described in **Section 6.1**.

Added (Section 6.1, page 13):

Core Timing Calibration. We calibrate the 2-GHz in-order FluxArch Core model against a 2-GB KickPi K3B with a 22-nm quad-core RK3562 Cortex-A53 [50, 55]. The native process is pinned to one 2-GHz core under stable thermal conditions; both platforms use identical source, GCC 13.3, -O2 -fno-tree-vectorize, seed, warm-up, and timed region. All six checksums match. Table 3 reports latency and signed error $100(T_{\text{gem5}} - T_{\text{K3B}})/T_{\text{K3B}}$; across the six tests, the mean and maximum absolute percentage errors are 6.7% and 13.5%, respectively. FluxArch's absolute throughput and speedups are obtained directly from the calibrated simulations.

References cited in this excerpt:

- [50] Rockchip Electronics Co., Ltd. [n.d.]. RK3562 Product Information. Rockchip product catalog. <https://www.rock-chips.com/a/en/products/> Accessed: 2026-08-23.
- [55] Shenzhen KICKPI Technology Co., Ltd. [n.d.]. KICKPI K3B Development Board. Official product page. <https://www.kickpi.com/product/k3b/> Accessed: 2026-08-28.

The six matched RK3562 calibration cases and their latency errors are reported in **Table 3**.

Added (Table 3, page 13):

Added

Table 3. Core timing calibration against the RK3562 Cortex-A53: (a) KickPi K3B board used as the silicon reference; (b) measured-versus-modeled latency.

(a) KICKPI K3B

Rockchip RK3562

Additional heatsink to ensure heat dissipation at 2GHz

Microbenchmark	K3B ns/operation	gem5 ns/operation	Error (%)
Integer, eight chains	6.18	6.52	+5.5
Branch, predictable	6.05	6.00	-0.8
Branch, pseudorandom	6.33	5.48	-13.5
Dependent load, 16 KiB	2.15	2.04	-5.0
Dependent load, 256 KiB	8.67	8.92	+2.8
Dependent load, 64 MiB	111.82	98.00	-12.4

Reviewer 3-Q1

- (3) **Performance with the calibrated model.** The performance results reported in **Section 6.2** and **Fig. 13(a,b)** already use the calibrated model described above; this revision makes the

calibration and RTL timing details explicit. The table below summarizes FluxArch’s speedup relative to each evaluated platform with workload orchestration. Each entry is the ratio of FluxArch throughput to the platform throughput for the same workload; the final column is the geometric mean of the five ratios.

FluxArch speedup relative to each platform ($\times$).							
Platform	CORAL2-P1	CORAL2-P2	CTS2	Homogeneous	XSbench	Geo. mean	
Gold 6430 $\times 2$	2.05	2.33	2.13	2.42	20.98	3.49	
EPYC 9654 $\times 2$	0.65	0.67	0.63	0.76	5.44	1.02	
FT 64-core	4.00	5.34	4.02	5.22	33.20	6.83	
NVIDIA A100	1.93	5.38	2.57	4.16	1.45	2.76	
NVIDIA H100	1.55	3.89	2.14	2.85	1.00	2.06	

In particular, the $6.83\times$ figure is the five-workload geometric-mean speedup over FT 64-core, not a uniform speedup across workloads or platforms. These results are supported by silicon-based core/memory calibration and RTL-derived Engine timing.

3.2 Question 2

The energy claim excludes DRAM, and the effect is not bounded.

The paper reports large energy-efficiency gains but excludes off-chip DRAM power. MCPT is memory-bound. The paper’s own data (Fig. 3(d)(e)) shows the working set exceeds on-chip capacity and forces frequent DRAM access. DRAM energy is therefore likely a large share of system energy. The on-chip scope is disclosed, which is fair. The problem is that the impact is never quantified. A system-level energy result could be materially smaller. As written, the energy advantage is an architectural-efficiency claim, not a system claim, and the gap between the two is unknown.

We agree that the original on-chip-only comparison could not support a system-level energy claim for a memory-bound workload. We changed the evaluation to include co-packaged memory through platform telemetry and off-package DDR5 through a common traffic-based DRAMPower model, and we vary the modeled DDR5 component over a $0.5\times-2\times$ range to assess sensitivity to missing command-level detail. The conservative process-scaled geometric-mean DRAM-inclusive energy-efficiency advantage remains $1.77\times$ over H100 and $6.65\times$ over Intel.

We now explain the choice of the $0.5\times-2\times$ range in **Section 6.1**. Ghose et al. [19] measured DDR3L read currents averaging 54.1% below datasheet values for one vendor and data-pattern-dependent read-power differences of up to 91.6%. Shi et al. [56] report that DDR4 calibration reduced DRAMPower energy estimates by roughly half for most evaluated SPEC2006 workloads. These results motivate testing sizable modeling discrepancies, but do not establish an error bound for our DDR5 configuration. We therefore select reciprocal factors of 0.5 and 2 to test both overestimation and underestimation, with package power fixed; the range is a sensitivity-analysis choice, not a statistical confidence interval.

For process-scaled energy-efficiency bounds, we independently vary only the scaled FluxArch package power by $\pm 30\%$ and each modeled off-package DRAM component over $0.5\times-2\times$, while keeping throughput and measured baseline package power fixed. We recompute the performance/W ratio at every endpoint combination and report its minimum and maximum; external DRAM power is not subject to process-node scaling or its 30% margin.

The power-accounting boundary is revised in **Section 6.1** to include DRAM consistently across platforms.

Revised (Section 6.1, page 14):

We report DRAM-inclusive system power and energy. Package power includes cores, caches, NoCs, and integrated memory controllers; for FT 64-core, NVIDIA A100, and NVIDIA H100, the main memory is co-packaged (DDR4 or HBM) and is consequently covered by platform-level package/device telemetry. In contrast, the DDR5 DIMMs of Gold 6430 $\times 2$ and EPYC 9654 $\times 2$, and the DDR5 channels assumed for FluxArch, are physically off-package and are not covered by package-power readings. We therefore add a separately modeled DRAM component to these platforms,

avoiding asymmetric accounting that would include HBM/DDR4 power for the co-packaged-memory platforms but omit DDR5 power for the other platforms.

The traffic-based DRAM model and factor-of-two sensitivity analysis are added in **Section 6.1**.

Added (Section 6.1, pages 14–15):

For the DRAM model, we use DRAMPower 5 [10] with DDR5 timing and worst-case current parameters from the JEDEC DDR5 standard [28] and the Micron MT60B2G8HB-48B DDR5-4800 x8 datasheet [42]. FluxArch is modeled with 32 single-rank DDR5-4800 channels, each containing eight x8 DRAM devices. The model includes active-standby background power, refresh power, and read/write transfer power. The external-memory energy of Gold 6430 $\times 2$ and EPYC 9654 $\times 2$ is estimated using the same traffic-based DDR5 model and workload-specific memory traffic.

Because the simulation does not expose DRAM command traces or row-buffer events, we use a traffic-based DRAM energy estimate rather than claiming command-level precision. For each FluxArch workload, we derive read/write traffic from the memory-system statistics and use the simulated execution time. Prior DRAM measurement and calibration studies report substantial discrepancies between datasheet-based estimates and measured consumption [19, 56]. Motivated by these findings, we select a $0.5\times\text{--}2\times$ range for the nominal modeled DRAM power of every off-package DDR5 platform (Gold 6430 $\times 2$, EPYC 9654 $\times 2$, and FluxArch). This range is a sensitivity-analysis choice, not an empirically established DDR5 error bound or a statistical confidence interval.

References cited in this excerpt:

- [10] Karthik Chandrasekar, Christian Weis, Yonghui Li, Sven Goossens, Matthias Jung, Omar Naji, Benny Akesson, Norbert Wehn, and Kees Goossens. [n. d.]. DRAMPower: Open-source DRAM Power and Energy Estimation Tool. <https://github.com/tukl-msd/DRAMPower>. Version 5.0, accessed July 2026.
- [28] JEDEC Solid State Technology Association. [n. d.]. DDR5 SDRAM Standard. Technical Report JESD79-5. JEDEC. <https://www.jedec.org/standards-documents/docs/jesd79-5>
- [42] Micron Technology, Inc. 2023. 16Gb DDR5 SDRAM Addendum: MT60B4G4, MT60B2G8, and MT60B1G16, Die Revision A. Micron Technology, Inc. <https://www.micron.com/products/memory/dram-components/ddr5-sdram> Rev. D, DDR5-4800 x8 specifications; the evaluated configuration is MT60B2G8HB-48B:A.
- [19] Saugata Ghose, Abdullah Giray Yağlıkçı, Raghav Gupta, Donghyuk Lee, Kais Kudrolli, William X. Liu, Hasan Hassan, Kevin K. Chang, Niladrish Chatterjee, Aditya Agrawal, Mike O'Connor, and Onur Mutlu. 2018. What Your DRAM Power Models Are Not Telling You: Lessons from a Detailed Experimental Study. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 2 (2018), 41 pages. doi:10.1145/3224419
- [56] Xinyu Shi, Dina Ali Abdelhamid, Thomas Ilsche, Saeideh Alinezhad Chamazcoti, Timon Evenblij, Mohit Gupta, and Francky Catthoor. 2025. Calibrating DRAMPower Model for HPC: A Runtime Perspective from Real-Time Measurements. *arXiv:2411.17960v3*. <https://arxiv.org/abs/2411.17960v3> Revised September 3, 2025.

The energy-efficiency comparison and conservative lower bounds are updated in **Section 6.2** to include DRAM power.

Original (Section 6.2, pages 14–15):

Iso-Process Energy Efficiency. As shown in Fig. 13(b), the energy efficiency advantage is even more pronounced. By comparing projected iso-process nodes, we isolate the architectural contributions. According to Section 6.1, the power projection error is 30%. Therefore, we incorporate this margin into our energy efficiency evaluation, which is represented as error bars in our results. Without the workload orchestration, the energy efficiency of FluxArch, mainstream server CPUs and high-end GPUs is at odds with each other. With the orchestration enabled, optimizing data locality and minimizing cross-tile traffic, FluxArch demonstrates a superior geometric mean energy efficiency improvement from $3.98\times$ (vs. FT 64-core, $5.69\times$ discounted by 30%) to $9.84\times$ (vs. Gold 6430 $\times 2$, $14.06\times$ discounted by 30%) even accounting for the potential projection error of pessimistic assumption. We also report the energy efficiency of the raw, unscaled data based on the 22 nm technology node. Even in this case, FluxArch still achieves an energy efficiency improvement of $1.17\times$ (vs. NVIDIA H100) to $4.59\times$ (vs. Gold 6430 $\times 2$).

Revised (Section 6.2, pages 15–16):

DRAM-Inclusive Energy Efficiency. Fig. 13(c,d) reports DRAM-inclusive power and energy efficiency under the sensitivity bounds defined in Section 6.1. Process-scaled results separate architectural effects from process scaling while accounting consistently for external memory.

In the process-scaled comparison, FluxArch improves geometric-mean DRAM-inclusive efficiency by $2.85\times$ over NVIDIA H100 to $10.58\times$ over Gold 6430 $\times 2$.

The sensitivity analysis preserves the process-scaled conclusion: across the combined DRAM and technology-scaling bounds, the geometric-mean improvement with orchestration remains $1.77\times$ over NVIDIA H100 and $6.65\times$ over Gold 6430 $\times 2$ at the conservative lower bound.

The energy-efficiency claim is updated in **Section 9** to reflect the process-scaled, DRAM-inclusive results.

Original (Section 9, page 20):

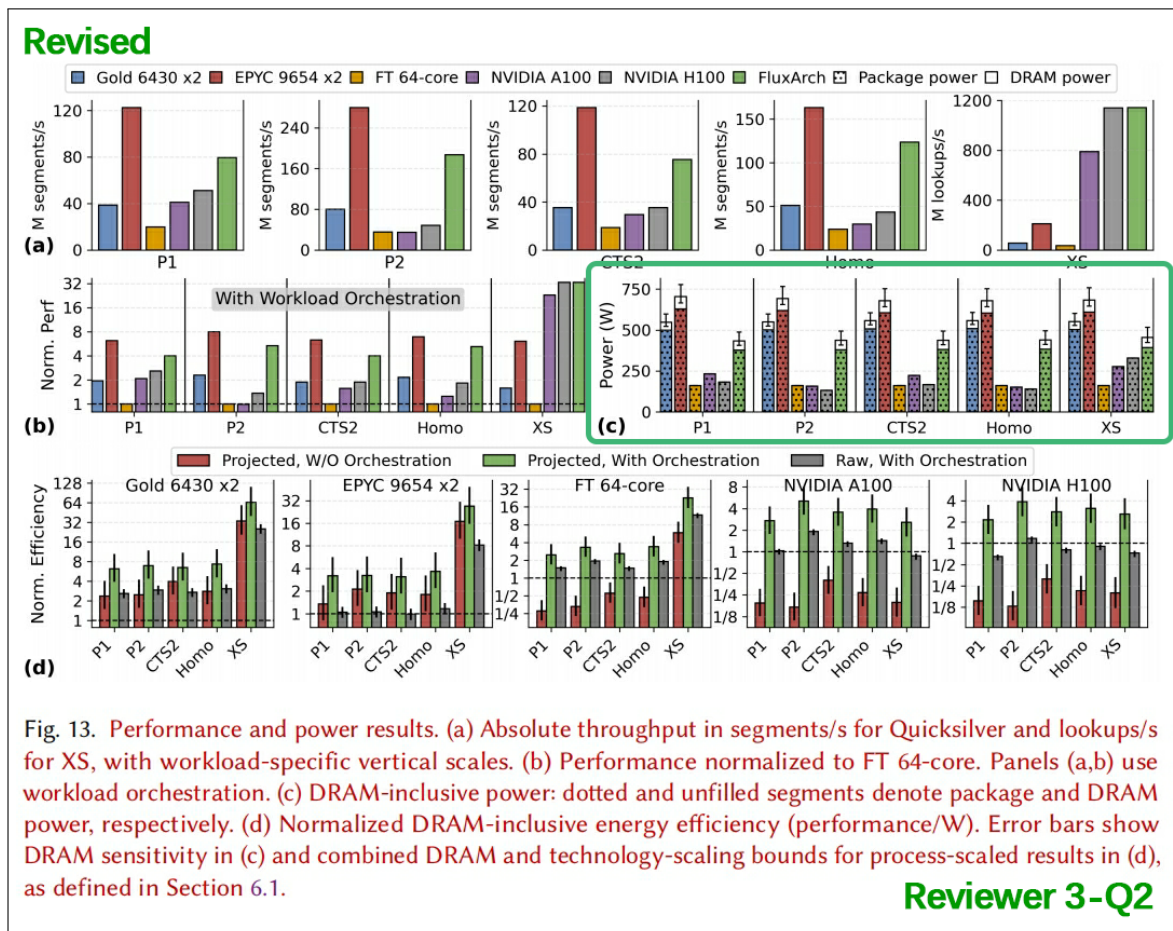
We introduced FluxArch, a thousand-core architecture that addresses the performance bottlenecks of MCPT through a large-scale MIMD design and specialized dataflow engines. By co-optimizing hardware topology and spatial decomposition, FluxArch overcomes irregular memory access limitations and delivers geometric mean speedups of up to $1.02\times$ – $6.83\times$ and energy efficiency improvements of up to $3.98\times$ – $9.84\times$ over traditional processors.

Revised (Section 9, page 23):

We introduced FluxArch, a thousand-core architecture that addresses the performance bottlenecks of MCPT through a large-scale MIMD design and specialized dataflow engines. **It also delivers process-scaled DRAM-inclusive energy-efficiency improvements of $2.85\times$ – $10.58\times$.**

The DRAM-inclusive power and energy-efficiency plots are updated in **Figure 13(c,d)**. Panel (c) is newly added; panel (d) revises the original energy-efficiency plot.

Revised (Figure 13, page 16):



3.3 Question 3

The baseline comparison has two specific gaps.

One benchmark (EMCP) is an event-based proxy, and the orchestration is applied to the CPU baselines, so the baselines are not naive. Two gaps remain. Particle-sorting MCPT methods are not evaluated, despite being a known GPU optimization. The comparison against prior specialized MCPT architectures is qualitative only (Table 4); there is no quantitative speedup against those designs. Without these, the claim of superiority over the state of the art is incomplete.

We sincerely thank you for your helpful and constructive suggestion. We address the two baseline gaps separately.

- (1) **Particle-sorting GPU baseline.** The GPU baseline now includes the particle-sorting optimization rather than comparing only against an unsorted implementation. We evaluated XSBench CUDA variants K4–K6 on both A100 and H100 with sampling, sorting, lookup, and reduction included in the timed and power-measurement interval, and selected the best end-to-end variant, K6. The strengthened H100 baseline reaches 1.140 billion lookups/s, nearly matching FluxArch’s 1.143 billion lookups/s; the five-workload geometric-mean speedup of FluxArch over H100 is consequently $2.06\times$.

The sorting-enabled GPU baseline selection and end-to-end timing/power boundary are specified in **Section 6.1**.

Added (Section 6.1, page 14):

GPU Particle-Sorting Baseline. For the GPU XSBench baseline, we evaluate the sorting-enabled CUDA variants K4–K6, which group cross-section lookups by material, fuel category, and/or particle energy to reduce warp divergence and improve memory locality. We use the variant with the highest end-to-end throughput on each GPU; K6, which combines material and energy sorting with material-specific kernels, is the best-performing evaluated variant on both NVIDIA A100 and NVIDIA H100. The reported runtime includes sampling, sorting, cross-section lookup, and result reduction, and the corresponding GPU-board power covers the same execution interval.

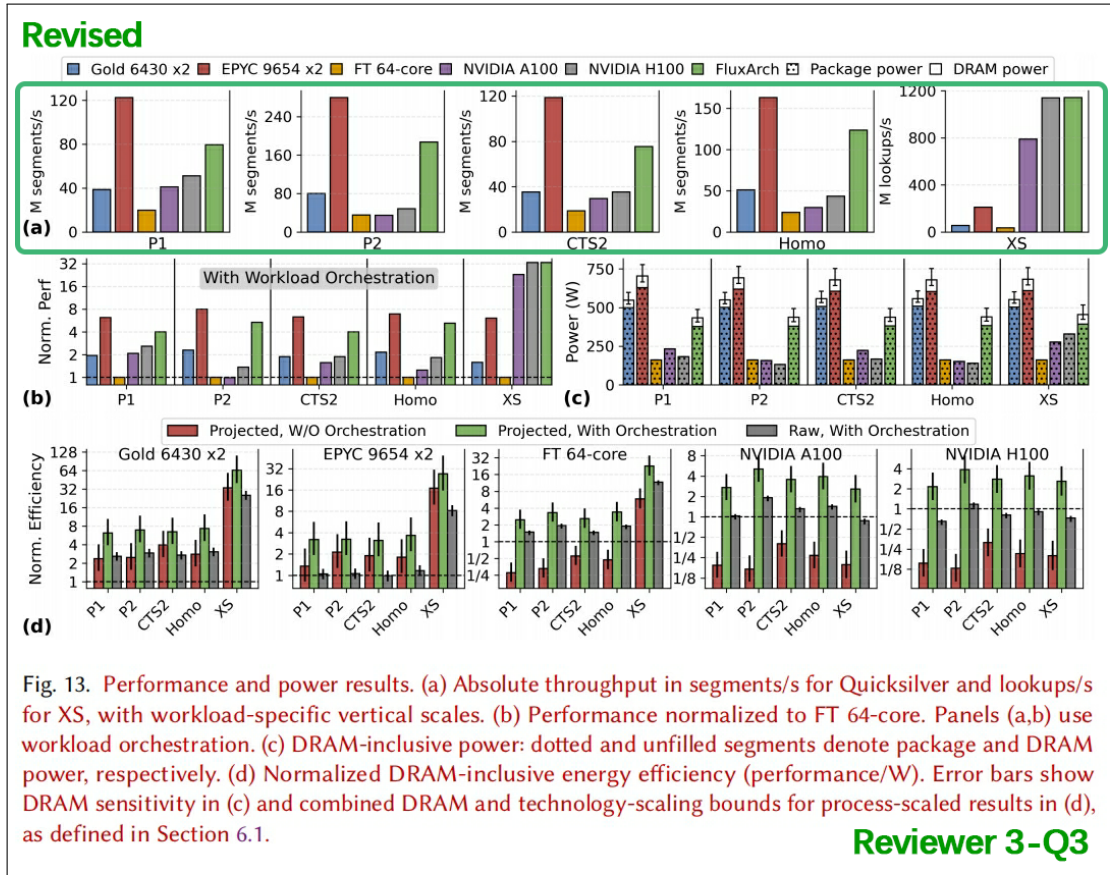
The K6 throughput and the resulting comparison with FluxArch are updated in **Section 6.2**.

Added (Section 6.2, page 15):

The selected K6 baseline reaches 788.63 million and 1.140 billion XS lookups/s on NVIDIA A100 and NVIDIA H100, respectively. Thus, FluxArch and NVIDIA H100 provide nearly identical XS throughput (1.143 versus 1.140 billion lookups/s), while FluxArch’s five-workload geometric-mean advantage over NVIDIA H100 is $2.06\times$.

The absolute throughput and normalized-performance plots are updated in **Figure 13(a,b)** using the sorting-enabled GPU baseline. Panel (a) is newly added; panel (b) revises the original normalized-performance plot.

Revised (Figure 13, page 16):



- (2) **Prior specialized MCPT architectures.** We added **Table 7** to compare FluxArch with the closely related designs of Fu et al. and Zhang et al., including their core design, hierarchy, cache and memory organization, methodology, workloads, and author-reported quantitative results. As clarified in **Section 8**, these results use different workload inputs, resources, baselines, and accounting boundaries. Fu et al.’s ratios are relative to a four-core Cortex-A15 model, whereas Zhang et al.’s scaling efficiency is relative to their own single-core configuration. They therefore cannot be converted into direct speedups over these designs.

A common reimplementaion would also require assumptions about implementation and modeling choices not fully specified in the papers. In particular, Zhang et al. do not report a process node or a power/area model. Reconstructing customized simulator configurations and adding such models could make the comparison depend on reconstruction choices as well as architectural differences. We added this limitation to **Section 8**, rather than presenting assumption-dependent estimates as a validated head-to-head comparison.

The comparison with Fu et al. and Zhang et al. is revised in **Section 8** to clarify the architectural differences.

Original (Section 8, page 19):

Recent microarchitectural studies have shifted toward specialized, area-efficient designs. Fu et al. pioneered the use of clustered small cores to improve energy-to-area ratios for MCPT workloads, a foundation subsequently extended by Zhang et al. to a 128-core cluster with optimized parallelization. Our FluxArch system achieves a significant leap forward by: (1) scaling the architecture to 1,024 cores using a hierarchical chiplet-based design; (2) integrating a dedicated execution engine to offload dominant kernels; and (3) proposing a topology-aware hardware-software co-design that transcends the scalability limits of prior clustered small-core architectures.

Revised (Section 8, page 22):

Specialized MCPT Architectures. Recent microarchitectural studies have shifted toward specialized, area-efficient designs. **Table 7 compares FluxArch with Fu et al. [18] and Zhang et al. [63] in core design, hierarchy, cache and memory organization, evaluation methodology, workloads, and author-reported results.** Fu et al. use simplified in-order cores to improve area and power efficiency, while Zhang et al. scale to 128 cores through 32-core cache-sharing clusters and OpenMP dynamic scheduling. FluxArch combines a 1,024-core hierarchy with a core-local Engine and topology-aware spatial mapping.

References cited in this excerpt:

- [18] *Siqing Fu, Tiejun Li, Jianmin Zhang, Sheng Ma, and Sheng Liu. 2021. MiniMCTAD: Minimalist Monte Carlo Transport Architecture Design. In 19th IEEE International Symposium on Parallel and Distributed Processing with Applications.*
- [63] *L. Zhang, T.-J. Li, and J.-M. Zhang. 2023. A 128-core scalable architecture for Monte Carlo application. Computer Engineering & Science / Jisuanji Gongcheng yu Kexue (2023), 590–598. Issue 4.*

The limits of comparing published results and reconstructing prior designs fairly are added in **Section 8**.

Added (Section 8, page 22):

The reported results use substantially different experimental conditions, including workload inputs, core counts, cache and memory configurations, reference baselines, and power-accounting boundaries. Fu et al. report performance, performance/W, and performance/area relative to a modeled four-core Cortex-A15 system. Zhang et al. report scaling efficiency relative to their own single-core configuration. FluxArch’s performance and energy-efficiency ratios use the evaluated CPU/GPU platforms as baselines, whereas its scaling efficiency is relative to one core. Thus, Table 7 places each work’s results in context rather than establishing cross-architecture speedups or an efficiency ranking.

A unified reimplementaion would also require resolving implementation and modeling choices not fully specified in the papers, rather than merely matching their listed hardware parameters. In particular, Zhang et al. do not report a process node or a power/area model. Reconstructing the customized simulator configurations and supplying missing power/area models would introduce additional assumptions that could affect the results, confounding architectural differences with reconstruction choices and weakening the fairness of a direct comparison.

The detailed architectural comparison and author-reported quantitative results are added in Table 7.

Added (Table 7, page 22):

Added

Table 7. Quantitative comparison with prior specialized MCPT architectures. Reported improvements use each work's own workload configuration and baseline and are not cross-paper normalized speedups.

Work	Fu et al.	Zhang et al.	FluxArch
Core	MiniMCTAD @ 2.2 GHz	HPI @ 2.2 GHz	FluxArch Core + dedicated Engine @ 2GHz
Hierarchy and Scale	1×8	4×32	$8 \times 8 \times 16$
Cache	512 kB shared L2	8 MB shared L2/cluster	4 MB shared L2/cluster
Memory	8-channel HBM2	8-channel HBM2	32-channel DDR5
Methodology	gem5+McPAT,28nm	gem5	gem5+McPAT+CACTI+DRAMPower,22nm
Benchmark	Quicksilver default	Quicksilver default	CORAL2-P1,CORAL2-P2,CTS2,Homogeneous,XSBench
Author-reported quantitative result	Relative to 4 Cortex-A15: 1.416x performance 4.56x perf. efficiency 3.01x area efficiency	70.1% scaling efficiency	Relative to various platforms: 1.02x - 6.83x performance 2.85x–10.58x energy efficiency (nominal process-scaled, DRAM-inclusive) 48% - 71% scaling efficiency

Reviewer 3-Q3